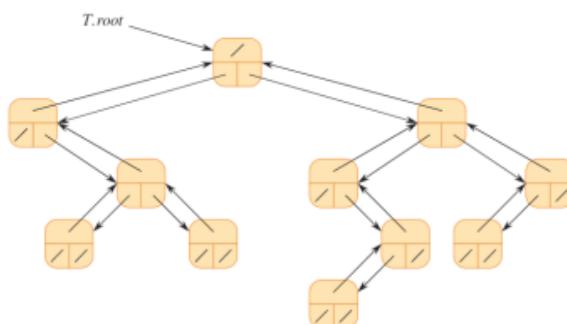


Appunti algoritmi e strutture dati

Alberi binari

Le linked lists vanno bene per rappresentare delle relazioni lineari (non tutte le relazioni sono lineari). In particolare ci concentriamo sul problema di rappresentare un albero costruito a partire da una linked data structure. Ogni nodo dell'albero è un oggetto e supponiamo che ogni nodo dell'albero contenga un attributo chiave (key). L'altro attributo d'interesse sono i puntatori che connettono ogni nodo agli altri. Un **albero binario** è un grafo non orientato rappresentato con i puntatori, caratterizzato dal fatto che ogni nodo ha al massimo due figli, uno destro e uno sinistro:

- Ogni nodo è definito con 3 puntatori, p , $left$, $right$, in cui p è il puntatore al nodo padre; il nodo $left$ è il puntatore al figlio sinistro e il nodo $right$ è il puntatore al figlio destro.
- Se il nodo x non ha figli sinistri, il puntatore $left$ è inizializzato a NIL . Analogamente per il caso destro.
- La radice ha il puntatore p inizializzato a NIL .
- La radice dell'intero albero è puntata dal parametro $.rt$.
- Se $.rt = NIL$ allora l'intero albero è vuoto.
- Le foglie dell'albero hanno i puntatori $left$ e $right$ inizializzati a NIL



È possibile estendere lo schema visto nell'albero binario a tutte le classi di alberi dove il numero di figlie di ogni nodo è al massimo un valore costante k . Per fare ciò è sufficiente sostituire i puntatori $left$ e $right$ con altri attributi come $child_1$, $child_2$, ..., $child_k$.

Problemi principali: Questo schema non funziona:

- In caso di variabile k non limitata superiormente.
- Se k è limitato da una costante molto grande ma la maggior parte dei nodi ha un numero basso di figli (spreco di memoria per i puntatori inutilizzati).

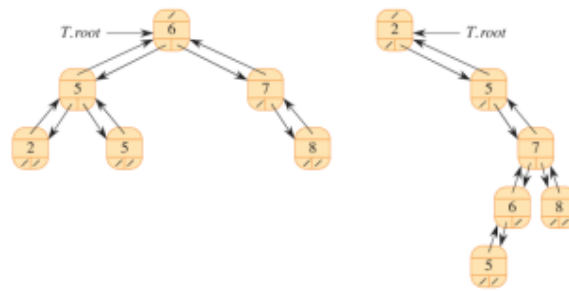
Rappresentazione *Left-child, right-sibling: Serve a generalizzare il concetto di albero, rappresenta un albero con un numero arbitrario di figli:

- Ha la caratteristica si usare solo $O(n)$ spazio per ogni albero di n nodi
- Ogni nodo è definito con un puntatore p al padre.
- Invece di avere un puntatore per ogni figlio, ogni nodo ha solo due puntatori
 - $leftchild$: figlio sinistro.
 - $rightibling$: fratello destro.
- Se il nodo non ha figli allora $leftchild = NIL$
- Se il nodo non ha fratelli destri allora $rightibling = NIL$

Albero binario di ricerca

Si tratta di un albero binario nel quale le chiavi (cioè le informazioni contenute nei nodi) sono sempre contenute in modo da soddisfare le proprietà dell'albero. **Proprietà dell'albero binario di ricerca:** Considerando un nodo x all'interno di un *albero binario di ricerca*:

- se il nodo si trova nella parte sinistra del sotto-albero di x , allora $.ke \leq x.ke$
- se il nodo si trova nella parte destra del sotto-albero di x , allora $.ke \geq x.ke$



In-order tree walk: Data la proprietà degli alberi binari di ricerca, è possibile visualizzare tutte le chiavi dell'albero in maniera ordinata grazie a un semplice algoritmo ricorsivo.

```
inorderTreeWalk(x)
if x!=NIL
    inorderTreeWalk(x.left)
    print x.key
    inorderTreeWalk(x.right)
```

L'algoritmo è così chiamato perché prima stampa tutti gli elementi di un sotto-albero partendo dalla sinistra della radice e concludendo alla sua destra.

Pre-order tree walk: Stampa la chiave prima dei valori di ciascun sotto-albero

Post-order tree walk: Stampa la chiave dopo i valori di ciascun sotto-albero

i Richiede $\Theta(n)$ in tempo per passare un albero binario di ricerca con n nodi. Dopo la chiamata iniziale la procedura chiama se stessa ricorsivamente 2 volte per ogni nodo nell'albero (una per ogni figlio)

Gli alberi binari di ricerca possono supportare gli algoritmi di richiesta di massimo, minimo, successore e predecessore esattamente come quello di ricerca. Queste operazioni richiedono un tempo $O(h)$ in un albero binario di ricerca di altezza h

Ricerca: Procedura di ricerca per un nodo con una chiave data in un albero binario di ricerca:

```
treeSearch(x,k)
if x == NIL || k == x.key
    return x
if k < x.key
    return treeSearch(x.left,k)
else return treeSearch(x.right,k)

iterativeTreeSearch(x,k)
while x != NIL && k != x.key
    if k < x.key
        x = x.left
    else
        x = x.right
return x
```

Dato un puntatore x alla radice di un sotto-albero e una chiave k , il metodo $treeSearch(x,k)$ ritorna un puntatore al nodo in cui la chiave è contenuta. Se tale nodo non esiste il metodo ritorna NIL . Per fare una ricerca "globale" della chiave richiamo $treeSearch(T.root, k)$. Questa procedura inizia alla radice dell'albero e traccia un percorso scendendo lungo l'albero. Per ogni nodo che viene incontrato si confronta la chiave k con la $x.key$ e se le due chiavi sono uguali la ricerca termina. Se la $x.key$ è più piccola di k la ricerca continua nel sotto-albero sinistro di x , altrimenti ($x.key$ più grande di k) si continua nel sotto-albero destro di x .

Massimo e minimo: Per trovare il minimo in un albero di ricerca binario basta seguire i puntatori ai figli sinistri di ogni nodo dalla radice fino a NIL (la procedura ritorna un puntatore al nodo contenente il minimo elemento). Per trovare il massimo si segue lo stesso principio scegliendo però il percorso fatto da sotto-alberi destri.

```
treeMinimum(x)
while x.left != NIL
```

```

    x = x.left
    return x

treeMaximum(x)
while x.right != NIL
    x = x.right
return x

```

Le proprietà degli alberi di ricerca binaria mi garantiscono la correttezza di questi metodi.

Successore e predecessore: Il successore di un nodo x è il nodo con la chiave più grande della $x.key$. Si definisce un successore il successivo nodo visitato in un in-order tree walk. La struttura degli alberi binari di ricerca mi permette di determinare il successore senza comparare le chiavi dei nodi. Tale procedura ritorna il puntatore al nodo successivo se esiste, altrimenti ritorna *NIL*.

```

treeSuccessor(x)
if x.right != NIL
    return treeMinimum(x.right)
else
    y = x.p
    while y != NIL && x == y.right
        x = y
        y = y.p
    return y

```

Tale procedura presenta 2 casi:

- il sotto-albero destro di x è non vuoto => il successore di x è il nodo più a sinistra nel sotto-albero destro di x .
- Il sotto-albero destro di x è vuoto e x ha un successore y => y è il più piccolo "genitore" di x il cui figlio (di y) è a sua volta un "genitore" di x . L'algoritmo per trovare il successore di un nodo richiede $O(h)$ tempo per un albero di altezza h in quanto segue o un *path-up* o un *path-down* dell'albero. Anche la procedura per trovare un predecessore, dato che è simmetrica a quella per trovare il successore, ha costo $O(h)$.

Inserimento: Sia la procedura di inserimento che quella di eliminazione causano delle modifiche all'albero (per rispettare le proprietà degli alberi binari di ricerca). La procedura per inserire un nodo dentro ad un albero binario di ricerca richiede un albero T e un nodo z la cui chiave deve essere già riempita con $z.left = NIL$ e $z.right = NIL$.

 **N.B.** la procedura modificherà T e alcune proprietà di z

```

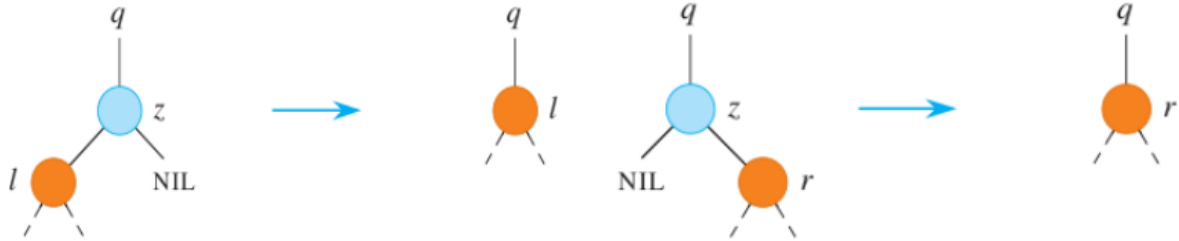
treeInsert(T,z)
x = T.root
y = NIL
while x != NIL
    y = x
    if z.key < x.key
        x = x.left
    else
        x = x.right
z.p = y
if y == NIL
    T.root = z
elseif z.key < y.key
    y.left = z
else
    y.right = z

```

La procedura inizia alla radice dell'albero e il puntatore x traccia un percorso discendente cercando un *NIL* da rimpiazzare con il nodo z . Questo algoritmo mantiene il puntatore come "genitore" di x . Il while fa sì che i due puntatori (x e y) si spostino verso il basso andando a destra e a sinistra in base al confronto di $z.key$ e $x.key$, finché x non diventa *NIL* (questo *NIL* occupa la posizione dove verrà posizionato il nodo $z = NIL$ è un attributo destro o sinistro del nodo che diventerà "genitore" di z , oppure è la $T.root$ se T è vuoto). Anche la procedura di inserimento costa $O(h)$ in tempo per un albero di altezza h .

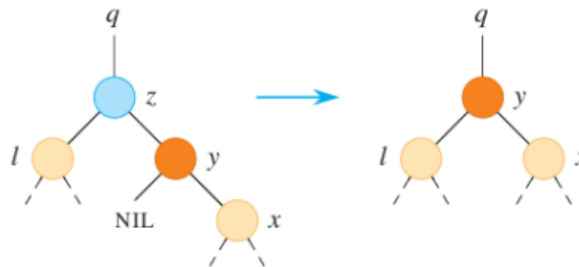
Eliminazione: Esistono 3 possibili strategie per eliminare un nodo da un albero di ricerca binaria:

1. z non ha "figli" (al massimo ne ha uno destro), quindi quando lo rimuovo modifico il "genitore" di z sostituendo quest'ultimo con NIL
2. z ha un "figlio" (in tal caso l'unico "figlio" è sinistro), quindi pongo il "figlio" di z al posto di questo e modifico conseguentemente il "genitore" di z

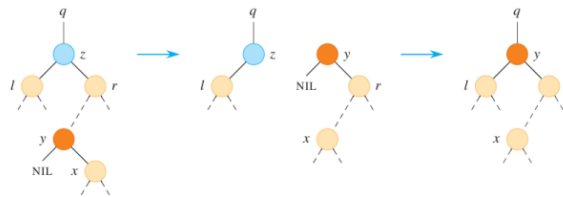


3. z ha due "figli", quindi viene trovato il suo successore y (nel sotto-albero destro di z) e si sposta y affinché prenda il posto di z . Il resto del restante sotto-albero destro di z diventa il nuovo sotto-albero destro di y e il sotto-albero sinistro di z diventa il sotto-albero sinistro di y . y , essendo il successore di z , non può avere un "figlio" sinistro e il "figlio" destro di y prende la posizione originaria di quest'ultimo (il resto del sotto-albero lo segue automaticamente). Di questo caso ci sono due possibili varianti:

- y è il "figlio" destro di z



- y si trova nel sotto-albero destro di z , ma non è il "figlio" destro



```

treeDelete(T,z)
  if z.left == NIL
    transplant(T,z,z.right)
  elseif z.right == NIL
    transplant(T,z,z.left)
  else
    y = treeMinimum(z.right)

    if y != z.right
      transplant(T,y,y.right)
      y.right = z.right
      y.right.p = y
    else
      transplant(T,z,y)
      y.left = z.left
      y.left.p = y

```

Come parte del processo di eliminazione di un nodo, ho la necessità di spostare interi sotto-alberi dell'albero binario di ricerca. Posso fare ciò con la subroutine *transplant*, la quale rimpiazza un sotto-albero "figlio" del proprio "genitore" con un altro sotto-albero. Nell'algoritmo quando rimpiazzo il sotto-albero con radice nel nodo u con il sotto-albero con radice nel nodo v , il "genitore" del nodo u diventa il genitore del nodo v .

```

transplant(T,u,v)
  if u.p == NIL
    T.root = v
  elseif u == u.p.left
    u.p.left = v
  else
    u.p.right = v

```

```
if v != NIL
    v.p = u.p
```



N.B. Sia l'algoritmo di inserimento che quello di eliminazione richiedono tempo $O(h)$ per alberi di altezza h

Red-black trees

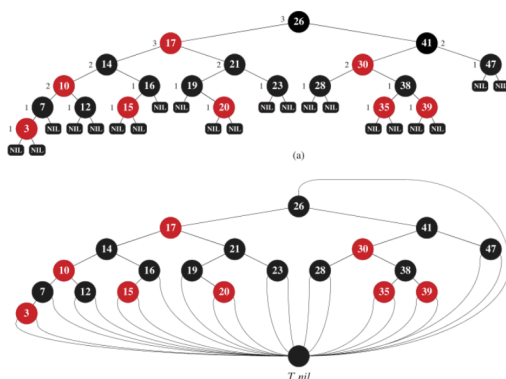
Il set di operazioni viste nella scorsa lezione sono veloci se l'altezza dell'albero di ricerca è piccola. Viceversa se l'altezza fosse un valore più grande le stesse operazioni sarebbero veloci tanto quanto una linked list. In questo contesto vengono introdotti gli alberi di ricerca bilanciati (**red-black trees**) che garantiscono delle operazioni che impiegano $O(\lg(n))$ di tempo nel caso peggiore.

Proprietà:

- Un red-black tree è un albero di ricerca binaria con un'ulteriore informazione per ogni nodo: il colore del nodo, il quale può essere o rosso o nero.
- Vincolando il colore del nodo ogni cammino semplice dalla radice alle foglie di un albero, ci si assicura che nessun cammino del genere sia più lungo del doppio di qualsiasi altro (dovrebbe voler dire che il cammino più lungo possibile è circa il doppio del cammino più corto possibile)
- L'albero è approssimativamente *bilanciato* (non ci sono foglie addossate a destra o a sinistra)
- L'altezza di un red-black tree con n chiavi è al massimo $2 \lg(n+1)$ che ha costo $O(\lg(n))$

Struttura:

- Ogni nodo dell'albero contiene gli attributi: colore, chiave, *left*, *right* e *p*
- Se un figlio o un genitore di un nodo non esistono, il puntatore corrispondente (attributo del nodo) contiene il valore *NIL* (questi valori possono essere pensati come i puntatori alle foglie)
- Un red-black tree è un albero di ricerca binaria che soddisfa le seguenti proprietà:
 1. Ogni nodo o è nero o è rosso
 2. La radice è nera
 3. Ogni foglia (*NIL*) è nera
 4. Se un nodo è rosso = entrambi i suoi figli sono neri
 5. Per ogni nodo, ogni cammino semplice discendente dal nodo fino alle foglie contiene lo stesso numero di nodi neri
- Si usa una singola sentinella per rappresentare *NIL*. La sentinella *.NIL* è un oggetto con gli stessi attributi di un normale nodo dell'albero. L'attributo colore della sentinella è il nero e gli altri attributi possono prendere valori arbitrari



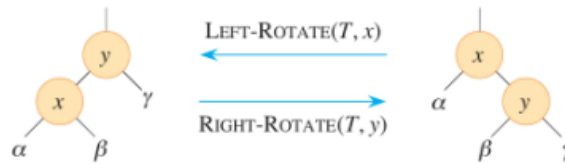
Nel primo dei due alberi i numeri accanto ai nodi stanno ad indicare quanti nodi neri ci sono da quel nodo fino alle foglie (per rispettare la proprietà 5)

La sentinella permette di trattare un figlio *NIL* di un nodo x come un nodo normale il cui padre sia x . Il numero di nodi neri in ogni cammino semplice discendente a partire (senza includere) un nodo x fino alle foglie è denotato come $bh(x)$. A partire dalle proprietà sopra definite si può dire che un black-height è ben definito. In generale un black-height di un red-black tree è un black-height della sua radice. Un red-black tree con n nodi interni ha altezza al massimo $2\lg(n+1)$

Basic operations: Le operazioni di base sono ricerca, minimo, massimo, successore, predecessore e richiedono tutti $O(\lg(n))$ di tempo.

i Come gli alberi binari di ricerca nei casi di insert e di delete viene modificata la struttura dell'albero venendo meno alle proprietà di questi alberi. Per sistemare questo aspetto le proprietà del colore e del puntatore di alcuni nodi devono cambiare. La struttura dei puntatori cambia attraverso rotazione, che è un'operazione locale in un albero di ricerca che preserva le proprietà degli alberi di ricerca binaria.

Esistono due tipi di rotazione: destra e sinistra. Entrambe richiedono $O(1)$ in tempo. Una rotazione sinistra di un nodo x , per esempio, richiede che il figlio destro del nodo (y) NON deve essere *.NIL*. La rotazione cambia il sotto-albero originariamente con radice in x scambiando il collegamento tra x e y (come nell'immagine seguente).



Inserimento: Per inserire un nodo in un red-black tree con n nodi interni in tempo $O(\lg(n))$ e mantenere le proprietà dell'albero è necessario modificare la procedura *treeInsert*. La procedura *RB_insert* inizia con l'inserimento del nodo z nell'albero T , come se fosse un normale albero di ricerca binaria, e colora questo nodo di rosso. Per garantire che le proprietà del red-black tree vengano preservate è necessario implementare la procedura *RB_insert_fixup* che ricolore i nodi e applica la rotazione.

```
RB_insert (T, z)
x = T.root
y = T.NIL
while x != T.NIL
    y = x
    if z.key < x.key
        x = x.left
    else
        x = x.right
z.p = y

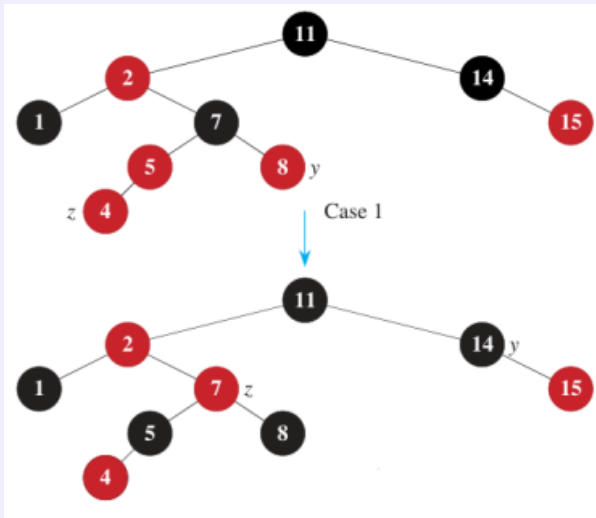
if y == T.NIL
    T.root = z
elseif z.key < y.key
    y.left = z
else
    y.right = z
z.left = T.NIL
z.right = T.NIL
z.color = RED
RB_insert_fixup
```

Differenze tra *treeInsert* e *RB_insert*:

- Tutte le istanze di *NIL* in *treeInsert* sono rimpiazzate da *.NIL*
- *z.left* e *z.right* sono importati a *.NIL* per mantenere la struttura dell'albero
- Alla fine il nodo z viene colorato di rosso
- Per evitare di violare le proprietà del red-black tree viene alla fine chiamato *RB_insert_fixup* (ogni nodo o è rosso o è nero, le foglie sono nero, il numero di nodi neri è lo stesso in ogni cammino semplice) => uniche proprietà che possono essere violate sono: radice deve essere nera e un nodo rosso non può avere un figlio rosso

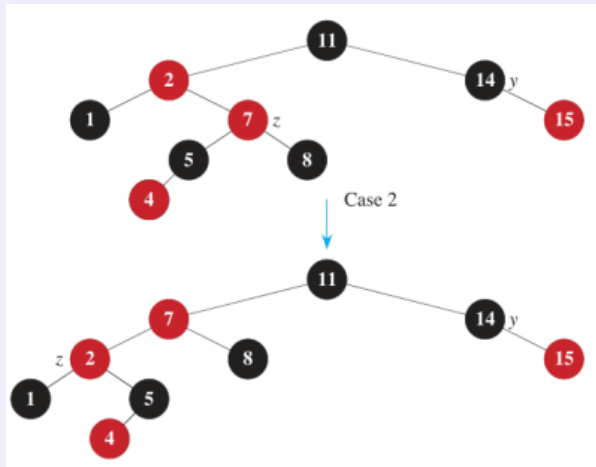
i Ci riferiamo allo zio come il nodo che è il figlio (di qualsiasi colore) di un nodo padre (nero)

1. y , lo zio di z , è rosso: quando sia $z.p$ sia y sono rossi. Poiché $z.p.p$ è nero, questa proprietà può essere trasferita di un livello in basso sia a $z.p$ sia a y così da sistemare il problema di avere sia z che $z.p$ rossi

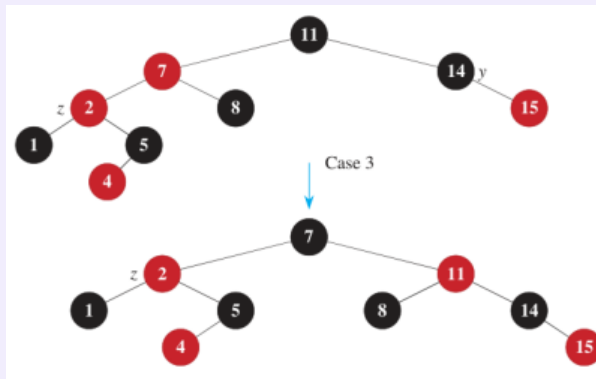


2. y , lo zio di z , è nero:

- y è nero e z è un figlio destro $\rightarrow$ una rotazione sinistra su z trasforma la situazione nel prossimo caso



- y è nero e z è un figlio sinistro $\rightarrow$ cambia alcuni colori e applica una rotazione destra su $z.p.p$, il quale preserva la proprietà di non avere figli rossi (essendo un nodo rosso)



Hash table

Una hash table è una generalizzazione di un array. In un array gli elementi vengono immagazzinati con una chiave k alla posizione k . Data la chiave k , per trovare l'elemento la cui chiave è k , mi basta guardare alla k -esima posizione dell'array (*direct addressing*). Quando non voglio allocare un array con una posizione per ogni possibile chiave uso una hash table. Uso questa struttura anche nel caso in cui il numero di chiavi immagazzinate è più piccolo del numero delle possibili chiavi. Quando si parla di hash table ci si riferisce ad un array che tipicamente usa una dimensione proporzionale al numero di chiavi che vanno immagazzinate. Per

questo motivo se ho la chiave k non uso questo valore come indice nell'array, invece definisco una *hash function* (funzione di k) e usiamo il suo valore per indicizzare l'array.

Direct-address tables	Hash tables
Mantiene un set dinamico	Richiede meno spazio di un array dato che può succedere che l'insieme di chiavi usate sia più piccolo dell'insieme di tutte le chiavi possibili
Ogni elemento ha una chiave presa dall'insieme universo $= \{0 \dots -1\}$	Può ridurre la richiesta di storage fino a $\Theta()$, con K l'insieme di chiavi effettivamente usate
Non esistono due elementi con la stessa chiave	Può ancora richiedere (nel caso medio, non nel peggiore) $O(1)$ in tempo per la ricerca
Se c'è un elemento x con chiave $k \Rightarrow T[k]$ contiene il puntatore a x	Si usa una funzione h per immagazzinare l'elemento nello slot $h(k)$

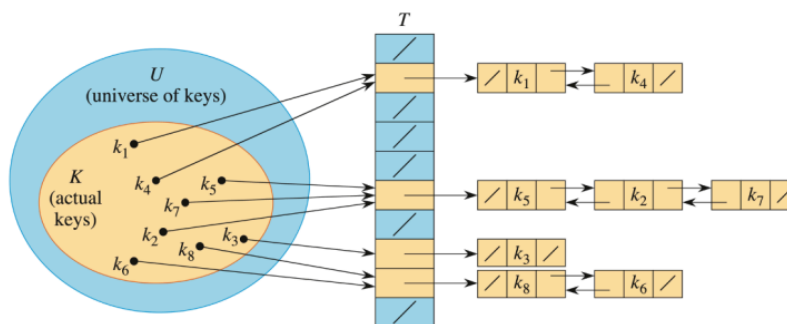
Info

- chiamiamo h la **hash function** e T la **hash table**
- $h: \rightarrow \{0, \dots, -1\}$, così $h(k)$ è uno slot in T

⚠ Quando due o più chiavi puntano allo stesso slot si ha una collisione:

- può accadere quando ci sono più chiavi che slot ($|K| > |T|$)
- dato un set K di chiavi con $|K| < |T|$ potrebbe come potrebbe non accadere (accade sicuramente se $|K| = |T|$)

Hashing indipendente e uniforme: Idealmente $h(k)$ verrebbe scelta casualmente e indipendentemente da $\{0, \dots, -1\}$. Una volta scelta $h(k)$ ogni valutazione seguente di $h(k)$ deve ridare lo stesso risultato. L'idea che la hash function sia indipendente, uniforme o casuale è teoretico e non può essere implementato nella pratica (si usa per analizzare il comportamento della hash function e per trovare delle approssimazioni pratiche alla forma ideale)



Collision resolution by chaining: Come un divide-et-impera non ricorsivo posso usare la hash function per dividere gli n elementi in maniera casuale in m sottoinsiemi, ognuno dei quali con circa n/m elementi. Successivamente posso gestire ciascun sottoinsieme in maniera indipendente come una linked list. Nei vari casi delle operazioni di inserimento, ricerca e cancellazione abbiamo:

- *insertion* = il caso peggiore richiede $O(n)$ in tempo
- *search* = il caso peggiore richiede una quantità di tempo proporzionale alla lunghezza della lista di elementi in ogni slot
- *deletion* = dato un puntatore x all'elemento da cancellare, in questo modo non devo cercare tale elemento

Data una chiave quanto ci metto a trovare l'elemento con quella chiave o per determinare che non c'è nessun elemento con quella chiave? Posso sfruttare un'analisi in termine di *fattore di carico*: $= n/m$ con:

- n = numero di elementi nella tabella
- m = numero di slot nella tabella = numero di linked lists (possibilmente vuote)
- Il fattore di carico è il numero medio di elementi per ogni linked list
- Posso avere $< 1, = 1, > 1$

Il caso medio dipende da quanto bene la hash function distribuisce le chiavi tra gli slot. Il caso peggiore, invece, si ha quando tutte le n chiavi sono inserite nello stesso slot per esempio il caso peggiore della ricerca è $\Theta(n)$. Se mi concentro sulla performance del caso medio dell'hashing with chaining:

- Assumo *hashing indipendente e uniforme* = ogni elemento dato può essere distribuito in ognuno degli m slot
- L'*hashing uniforme e indipendente* è universale = la probabilità che ogni 2 chiavi distinte collidano è $\frac{1}{m}$
- $j=0,1,\dots,m-1$, si denota la lunghezza della lista j da n_j , in questo modo $n = n_0 + n_1 + \dots + n_{m-1}$
- Il valore atteso di n_j è $En_j = \frac{n}{m}$
- Assumo che la hash function richieda $O(1)$ in tempo per essere eseguita, in questo modo il tempo richiesto per cercare l'elemento di chiave k dipende dalla lunghezza $n_{h(k)}$ della lista $h(k)$

Teorema: una ricerca senza successo richiede nel caso medio $\Theta(1+)$ in tempo (dimostrazione si basa sul fatto che una ricerca senza successo arriva in fondo alla lista di lunghezza attesa e a questo bisogna aggiungere il tempo per eseguire la hash function). Anche una ricerca con successo nel caso medio richiede $\Theta(1+)$ in tempo (**Teorema**).

Cosa rende buona una hash function? Idealmente la hash function soddisfa l'*hashing indipendente e uniforme*: ogni chiave ha la stessa probabilità delle altre chiavi di essere distribuita in ognuno dei m slot. Nella pratica, però, non è possibile soddisfare tale assunto siccome non si conosce in anticipo la distribuzione di probabilità che le chiavi siano estratte, e le chiavi potrebbero non essere estratte indipendentemente (se si conosce la distribuzione delle chiavi si può avere un vantaggio da ciò) Nella pratica le hash function assumono che le chiavi siano alternativamente:

- Un intero non negativo e corto che rientri in una *machine word* (32 o 64 bit di solito)
- Un vettore corto di interi non negativi, ciascuno dei quali rientra in un limite definito

Static hashing: Si tratta di una hash function fissa per la quale la casualità viene solo dalla "speranza per la distribuzione" delle chiavi:

- *Division method:* $h(k) = k \bmod m$. Il vantaggio risiede nel fatto che è veloce e richiede solo un'operazione di divisione. Una buona scelta per m è un numero primo non troppo vicino ad una potenza esatta di 2

$$m = 20 \text{ e } k = 91 \quad h(k) = 1 \bmod 20 = 11$$

- *Multiplication method:* si sceglie la costante A nel range $0 < A < 1 \rightarrow$ moltiplico la chiave k per $A \rightarrow$ estraggo la parte frazionaria da $kA \rightarrow$ moltiplico la parte frazionaria per $m \rightarrow$ prendo il *floor* del risultato. Questo metodo è più lento del *division method*, ma il valore di A non è critico (può essere scelto indipendentemente da A)

 $h(k) = (kA \bmod 1)$, dove $kA \bmod 1 = kA - \lfloor kA \rfloor$ = parte frazionaria di kA

Hashing casuale: Un ipotetico avversario malevolo, conoscendo la funzione hash, potrebbe scegliere chiavi che collidano tutte nello stesso slot, producendo il caso pessimo. Qualsiasi funzione hash statica è vulnerabile a questo attacco. La soluzione sta nello scegliere la funzione hash **casualmente**, indipendentemente dalle chiavi. Un caso particolare è l'**hashing universale**, che garantisce buone prestazioni medie quando si usa il chaining, indipendentemente dalle chiavi. Si consideri una collezione finita di funzioni hash che mappi l'universo di chiavi nel range $\{0,1,\dots,m-1\}$. È universale se per ogni coppia di chiavi $k_1, k_2 \in U$, il numero di funzioni hash $h \in H$ con $h(k_1) = h(k_2)$ è $\frac{|H|}{m}$. In altre parole è universale se, con una funzione hash h scelta casualmente da H , la probabilità di una collisione tra due chiavi distinte non supera l' $\frac{1}{m}$ di probabilità di scegliere due slot casuali e indipendenti

Proprietà hashing casuale: Si possono ottenere limiti teorici sulle collisioni e sulle prestazioni attese. Considero una famiglia di hash function su un dominio U e con range $\{0,1,\dots,m-1\}$, le chiavi appartengono a U , i numeri degli slot sono in $\{0,1,\dots,m-1\}$, e una hash function h è presa casualmente da H :

- Uniforme: la probabilità di prendere h tale che $h(k) = i$ è $\frac{1}{m}$
- Universale: per ogni chiave distinta k_1, k_2 , la probabilità che $h(k_1) = h(k_2)$ è al massimo $\frac{1}{m}$
- d -universale: per ogni chiave distinta k_1, k_2 , la probabilità che $h(k_1) = h(k_2)$ è al massimo $\frac{1}{m^d}$
- d -indipendente: per ogni chiave distinta k_1, k_2 e ogni slot $i_1, i_2, \dots, i_d$, non necessariamente distinto, la probabilità che $h(k_i) = i_i$ è $\frac{1}{m^d}$

Hashing basato su teoria dei numeri: Alcune famiglie di funzioni hash utilizzano proprietà della teoria dei numeri per garantire buone distribuzioni. Scelto un numero primo p abbastanza grande da permettere che ogni possibile chiave appartenga a $\{0,1,\dots,p-1\}$ e si assuma che $p > m$. Si denoti $U = \{0,1,\dots,p-1\}$, $V = \{0,1,\dots,p-1\}$. Dato ogni $a \in U$ e ogni $b \in V$, si definisce $h_{ab}(k) = ((ak + b) \bmod p) \bmod m$. Dato p e m , la famiglia di hash function è

$ab = \{h_{ab} : a \in_p, b \in_p \text{ (ognuna delle quali mappa da } p \text{ a } p)\}$. Questa famiglia di hash functions contiene $p(p-1)$ funzioni, una per ogni combinazione di a e b .

Teorema: la famiglia $_p$ definita sopra di hash functions è universale

Open addressing: Idea principale:

- Tutti gli elementi sono memorizzati **direttamente nella tabella**.
- Ogni slot contiene un elemento oppure NIL.
- La tabella può riempirsi, ma il fattore di carico non può superare 1.

Inserimento:

1. Calcolare la prima posizione scelta.
2. Se è libera, inserire l'elemento.
3. Se è occupata, calcolare la seconda posizione.
4. Continuare finché si trova uno slot libero.

Ogni elemento ha una propria sequenza di preferenze.

```
Hash-insert(T,k)
i = 0
repeat
    q = h(k,i)
    if T[q] == NIL
        T[q] = k
        return q
    else
        i = i+1
until i == m
error "hash table overflow"
```

Ricerca: Si segue la stessa sequenza di probe dell'inserimento, ma se si incontra uno slot vuoto, si conclude che l'elemento non è presente. Il vantaggio di tale opzione è che non servono puntatori (a differenza del chaining) e che lo spazio risparmiato può essere usato per aumentare la dimensione della tabella (slide 30 pdf 11 per avere più specifiche nella ricerca della chiave k).

```
Hah-Search(T,k)
i = 0
repeat
    q = h(k,i)
    if T[q] == k
        return q
    i = i+1
until T[q] == NIL or i == m
return NIL
```

Cancellazione: La cancellazione è più delicata rispetto al chaining. Non si può semplicemente mettere NIL, perché si interromperebbero le sequenze di probe = è l'ordine dei bucket (posizioni) controllati nella tabella hash fino a quando l'elemento viene trovato o viene trovata una posizione vuota (un probe o sonda è un tentativo di inserire o cercare un elemento in una specifica posizione della tabella). Si usano marcatori speciali (slot DELETED) al posto di NIL per marcare uno slot come vuoto durante la cancellazione.

- Search dovrebbe trattare DELETED come se lo slot contenesse una chiave che non corrisponde a quella che viene cercata
- Insertion dovrebbe trattare DELETED come se lo slot fosse vuoto, così da poter essere riusato

Lo svantaggio di usare DELETED è che ora il tempo di ricerca non dipende più dal fattore di carico. Un caso speciale di open addressing, chiamato linear probing, evita di avere slot marcati con DELETED

Sequenze di Probe: La situazione ideale è quella di *independent uniform permutation hashing*: ogni chiave può avere ognuna delle permutazioni di $\langle 0, 1, \dots, -1 \rangle$. Siccome è complesso implementare una vera independent uniform permutation hashing, posso approssimarla nei modi seguenti:

- **Double hashing:** Si usano due funzioni hash ausiliari h_1, h_2 e probe number i : $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Il primo probe va nello slot $h_1(k)$, successivamente i probe sono compensati dallo slot precedente da $h_2(k)$ mod m la probe sequence dipende dalle chiavi in 2 modi. È necessario che $h_2(k)$ sia coprimo di m (nessun fattore comune a parte 1) per garantire che la probe sequence sia una permutazione di $\langle 0, 1, \dots, -1 \rangle$. Questa soluzione

riduce clustering (formazione di gruppi di elementi consecutivi nella tabella) e migliora la distribuzione.

- Si può scegliere m affinché sia una potenza di 2 e h_2 affinché produca sempre un numero dispari
- Si può fare in modo che m sia primo e avere $1 < h_2(k) < m$
- **Linear probing:** È un caso speciale del double hashing. Data una funzione hash ausiliaria h , la probe sequence inizia allo slot $h(k)$ e continua attraverso la tabella (da slot $m-1$ allo slot 0). Data una chiave k e un probe number i , ($0 \leq i < m$): $h(k, i) = (h(k) + i) \bmod m$. Il probe iniziale determina tutta la sequenza ogni m sequenza possibile. Semplice ma soffre di **primary clustering**.

Analisi dell'open addressing: Un'analisi in termini di fattore di carico implica che:

- Si assuma che la tabella non si riempie mai, quindi si ha sempre $0 \leq n < m \leq 1$
- Si assuma un independent uniform permutation hashing
- Nessuna cancellazione
- In una ricerca con successo, ogni chiave è ugualmente probabile che sia cercata

Teorema: il numero atteso di probe in una ricerca senza successo è al massimo $\frac{1}{1-\alpha}$

Corollario: il numero atteso di probe da inserire è al massimo $\frac{1}{1-\alpha}$

Teorema: il numero atteso di probe in una ricerca con successo è al massimo $\frac{1}{\alpha} \ln(\frac{1}{1-\alpha})$ (su pdf ci sono le dimostrazioni di tutti e 3)

Considerazioni pratiche: Le CPU moderne influenzano il comportamento dell'hashing.

1. Gerarchie di memoria:

- Le cache sono piccole e veloci.
- Organizzate in cache line (es. 64 byte contigui).
- È più veloce riusare una cache line che accedere alla memoria principale.
- Iterare su dati contigui è vantaggioso.

2. Set di istruzioni avanzati: Alcune istruzioni hardware (usate per crittografia) possono essere sfruttate per calcolare hash efficienti.

3. Linear Probing: Nel modello RAM teorico il linear probing può performare male. In presenza di cache gerarchiche:

- Funziona meglio perché probe consecutivi tendono a restare nella stessa cache line.

Sintesi:

Hashing diventa una funzione in due variabili $h(k, p)$ dove p indica il probe, ovvero il "tentativo" di inserimento nella hash table (in caso di collisione)

Hashing lineare: $\text{Indice} = h(k) + p \bmod m$ Facilmente "hackerabile" perché rende molto probabile che se c'è un blocco di elementi allocati il prossimo cada sul primo blocco libero

Hashing con moltiplicazione e somma: comunque facilmente hackerabili (ad es, $m=16$ se inserisco $26 + k$ ottengo comunque collisioni sull'indice 10)

Hashing quadratico: meno prevedibile

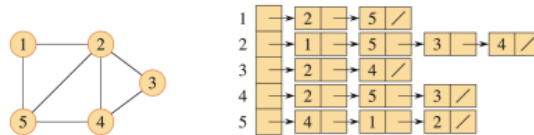
Grafi

Dato un grafo $G=(V, E)$, in pseudocodice si rappresenta il set di vertici con $G.V$ e il set degli archi con $G.E$.

- G può essere orientato o non orientato
- Due modi comuni per rappresentare i grafi per gli algoritmi sono:

1. **Liste di adiacenza:** array Adj di $|V|$ liste, una per vertice. La lista del vertice u ha tutti i vertici v in modo da avere che $(u, v) \in E$ (vale sia per grafi orientati che non orientati). In pseudocodice si denota tale array come un attributo ($G.Adj$)

For an undirected graph:



Per quanto riguarda l'espressione del tempo di esecuzione di un algoritmo, è spesso in termini sia di $|V|$ sia di $|E|$ (cardinalità).

Breadth-first search:

- In input ho un grafo $G=(V,E)$, sia orientato che non orientato, e un vertice sorgente $\in V$.
- In output ho:
 - $v.d$ = distanza (numero di archi più piccolo) da a a v per ogni $v \in V$
 - $v.\pi$ = è il predecessore di v nel cammino più breve da a . $(,v)$ è l'ultimo arco nel cammino più breve $\rightsquigarrow v$. Il sotto-grafo predecessore contiene gli archi $(,v)$ tali che $v.\pi =$. Il sotto-grafo predecessore forma un albero chiamato *breadth-first tree*.

Intuizione: i breadth-first tree espandono la frontiera tra i vertici scoperti e non scoperti uniformemente attraverso l'ampiezza della frontiera. A partire da a vengono scoperti i vertici in ondate

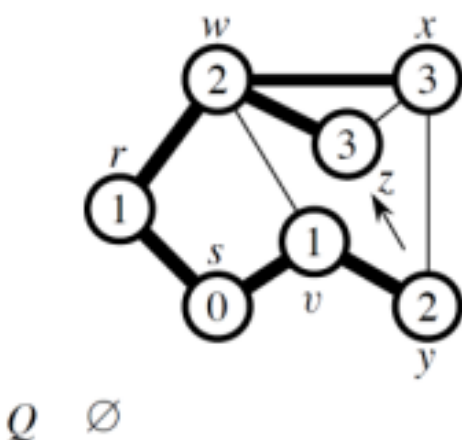
- Prima visita tutti i vertici distanti 1 arco da a
- Da qui visita tutti i vertici distanti 2 archi da a
- Etc.

Per mantenere il "fronte d'onda" si usa il metodo FIFO sulla coda :

- $v \in Q$ l'onda ha visitato v ma non è ancora uscito da v
- Q contiene i vertici a distanza k , e possibilmente alcuni vertici a una distanza $k+1$. Perciò in ogni momento Q contiene porzioni di due ondate consecutive

```

BFS(G,s)
for each vertex  $u \in G.V - \{s\}$ 
     $u.color = WHITE$ 
     $u.d = \infty$ 
     $u.\pi = NIL$ 
 $s.color = GRAY$ 
 $s.d = 0$ 
 $s.\pi = NIL$ 
 $Q = \emptyset$ 
ENQUEUE(Q,s)
while  $Q \neq \emptyset$ 
     $u = DEQUEUE(Q)$ 
    for each vertex  $v \in G.Adj[u]$  // cerca i vicini di  $u$ 
        if  $v.color == WHITE$  //  $v$  è stato scoperto?
             $v.color = GRAY$ 
             $v.d = u.d + 1$ 
             $v.\pi = u$ 
            ENQUEUE(Q,v) //  $v$  è sulla frontiera ora
     $u.color = BLACK$  //  $u$  è dietro la frontiera ora
  
```



- gli archi più spessi sono nel sotto-grafo predecessore
- le linee tratteggiate vanno ai vertici appena scoperti
- i vertici cerchiati 2 volte sono stati scoperti e sono in Q in attesa di essere visitati
- i vertici cerchiati con una linea più spessa sono stati scoperti, tolti dalla coda e visitati
- BFS potrebbe non raggiungere tutti i vertici e richiede in tempo $O(V+E)$
- $O(V)$ perché ogni vertice è messo in coda al massimo una volta
- $O(E)$ perché ogni vertice è estratto dalla coda al massimo una volta e l'arco (u,v) è esaminato solo quando u è estratto dalla coda. In questo modo ogni arco viene esaminato al massimo 1 volta se orientato, al massimo 2 volte se non orientato

Per stampare i vertici del cammino più corto da a a v :

```

if v == s
    print s
elseif v.parent == NIL
    print "no path from" s "to" v "exists"
else PRINT-PATH(G,s,v.parent)
    print v

```

```

BFS(G, s)
    ► GANCIO INIT – inizializzazione di tutti i nodi
    d(s) = 0; color(s) = gray
    Q = coda vuota; enqueue(Q, s)

    while Q ≠ ∅ do
        u = dequeue(Q)
        ► GANCIO DEQUEUE – stai elaborando u

        for ogni v ∈ Adj[u] do
            if color(v) == white then
                color(v) = gray
                d(v) = d(u) + 1
                π(v) = u
                ► GANCIO BIANCO – hai appena scoperto v
                enqueue(Q, v)
                ► GANCIO STOP – condizione di terminazione anticipata

        color(u) = black
        ► GANCIO POST – u è completamente elaborato

    ► GANCIO ESTERNO – for su tutti i nodi (grafi non connessi)

```

Depth-first search:

- *Input*: un grafo $G=(V,E)$ sia orientato che non orientato. Non viene dato nessun vertice sorgente
- *Output*:
 - 2 marcatori del tempo su ogni vertice utili per algoritmi che vedremo poi:
 - $v.d$ = momento della scoperta del vertice
 - $v.f$ = tempo finale
 - $v.\pi$ è il predecessore di v nella **depth-first forest** con più di un **depth-first tree**. Se $\pi(v) = u$ allora (u,v) è un ramo dell'albero

Questo algoritmo esplora ogni arco e inizia da vertici diversi se necessario. Appena viene scoperto un vertice inizia ad esplorare da questo (diversamente dal BFS che mette un vertice in coda così che venga esplorato successivamente). Man mano che il DFS progredisce, ogni vertice ottiene un colore:

- Bianco = vertice ancora non scoperto
- Grigio = vertice scoperto ma non finito (non si è finito di esplorare da quel vertice)
- Nero = vertice finito (è stato trovato tutto ciò che era raggiungibile da tale vertice)

Tempi di scoperta e di fine:

- Interi unici da 1 a $2|V|$
- Per ogni $v, v.d < v.f$

In altre parole $1 \leq v.d < v.f \leq 2|V|$

```

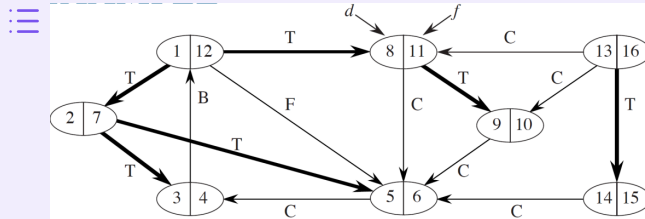
DFS(G)
    for each vertex u in G.V
        u.color = WHITE
        u.parent = NIL
    time = 0
    for each vertex u in G.V
        if u.color == WHITE
            DFS-Visit(G,u)
    la ricerca globale fa partire una ricerca locale su ogni vertice per esplorare tutto il grafo

```

```

DFS-Visit(G,u)
  time++ // white vertex u has just been discovered
  u.d = time
  u.color = GRAY
  for each vertex v in G.Adj[u] //explore each edge (u,v)
    if v.color == WHITE
      v.p = u
      DFS-Visit(G,v)
  time ++
  u.f = time
  u.color = BLACK //blacken u because it is finished

```

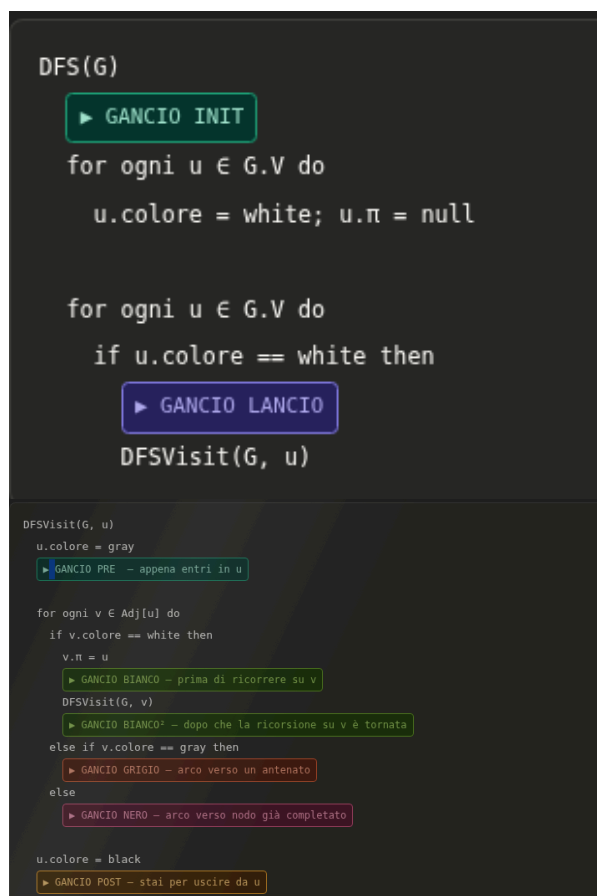


Tempo = $\Theta(V + E)$

- l'analisi è simile a quella per il BFS
- Θ e non solo O siccome è garantito che venga esaminato ogni vertice e ogni arco

Ogni depth-first tree è fatto da archi (v) così che sia grigio e v sia bianco quando (v) è esplorato

Struttura tipica di una DFS e DFSVisit:



Teorema (delle parentesi): per ogni v vale esattamente una delle seguenti:

1. $.d < .f < v.d < v.f$ oppure $v.d < v.f < .d < .f$ (cioè gli intervalli $.d, .f$ e $v.d, v.f$ sono disgiunti) e ne v è discendente dall'altro
2. $.d < v.d < v.f < .f$ e v è discendente da v (v è scoperto dopo e finito prima di v)
3. $v.d < .d < .f < v.f$ e v è discendente da v (v è scoperto dopo e finito prima di v)

Cioè $.d < v.d < .f < v.f$ non può accadere (v è sia scoperto che finito dopo)

Classificazione degli archi: posso sfruttare i tempi di inizio e di fine di ciascun nodo per classificare gli archi come:

- **Tree edge:** dentro la depth-first forest, trovato esplorando $(,v)$ (nodo viene scoperto per la prima volta attraverso questo arco)
- **Back edge:** $(,v)$ con v discendente da v (arco che collega un nodo con il suo antenato, quindi il tempo di fine del figlio è minore del tempo di fine dell'antenato)
- **Forward edge:** $(,v)$ con v discendente da v , ma non è un tree edge (arco che collega un nodo con il suo discendente magari già esplorato)
- **Cross edge:** ogni altro arco. Può andare in mezzo ai vertici dello stesso depth-first tree o in depth-first tree diversi

Stato di v	Condizione Temporale	Tipo di Arco
Bianco	-	Albero $\emptyset$
Grigio	-	Indietro (Ciclo!) $\emptyset$
Nero	$u.d < v.d$	Avanti $\emptyset$
Nero	$u.d > v.d$	Trasversale $\emptyset$

In un grafo non orientato, ci possono essere delle ambiguità siccome $(,v)$ e $(v,)$ sono lo stesso arco. Si classifica in base al primo tipo sopra descritto corrispondente

Topological sort: Un grafo orientato e senza cicli è buono per modellare i processi e le strutture che hanno un ordine particolare:

- $a \rightarrow b \rightarrow c$ e $b \rightarrow c \rightarrow a$
- Ma potrebbe avere a e b tali che ne $a \rightarrow b$ ne $b \rightarrow c$

Si può sempre ottenere un ordine totale da un ordine parziale (o $a \rightarrow b$ o $b \rightarrow a$ per ogni $a \neq b$). Difatti questo è ciò che fa un topological sort.

Dag: è un grafo orientato e aciclico. Un grafo G è aciclico un DFS di G non produce un arco nero (prova alla slide 21)

Topological sort di un dag: ordina linearmente i vertici così che se $(,v) \in E$ allora u appare da qualche parte prima di v :

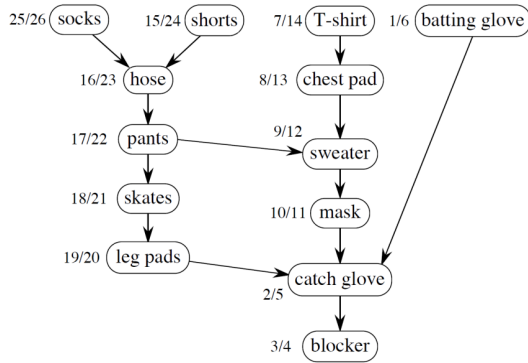
1. Chiama DFS(G) per valutare i tempi di fine $v.f$ per ogni vertice v
2. Quando ogni vertice è finito, lo inserisce all'inizio della linked list
3. Ritorna la linked list dei vertici

Non necessita di ordinare in funzione dei tempi di fine:

- Può far uscire i vertici appena sono finiti e capisce che voglio il *reverse* della lista
- Oppure mette i vertici all'inizio della linked list appena sono finiti. Appena finito la lista contiene i vertici ordinati con ordine topologico

Tempo: $\Theta(V + E)$

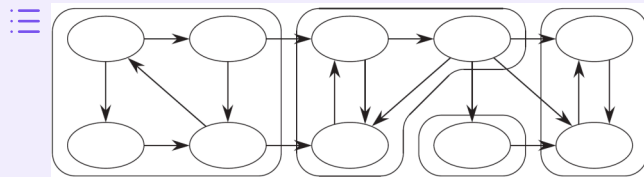
Let's run DFS on this example to find finish times.



Correttezza: è necessario mostrare che se $(,v) \in E$ allora $v.f < .f$. Quando l'arco $(,v)$ è esplorato, di che colore sono $.$ e v ?

- $.$ è grigio
- Anche v è grigio? No, perché in tal caso v sarebbe "antenato" di $(,v)$ è un arco nero contraddizione del lemma precedente (i dag non hanno archi neri)
- v è bianco? In tal caso diventa "discendente" di $.$. In base al teorema delle parentesi $.d < v.d < v.f < .f$
- v è nero? In tal caso v sarebbe già finito. Siccome sto esplorando $(,v)$. non è ancora finito, quindi $v.f < .f$

Componenti fortemente connesse: dato un grafo orientato $G = (V, E)$, una componente fortemente connessa (SCC) di G è il set massimale di vertici V così che per ogni $,v \in , \rightsquigarrow v$ e $v \rightsquigarrow ,$



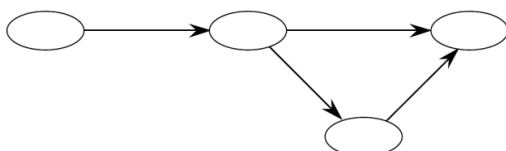
Nell'esempio: L'algoritmo usa G^T = trasposta di G :

- $G^T = (V, E^T), E^T = \{(\cdot, v) : (v, \cdot) \in E\}$
- G^T è G con tutti gli archi invertiti

i posso creare G^T in $\Theta(V + E)$ di tempo se uso le liste di adiacenza

Osservazione: G e G^T hanno gli stessi SCC ($.$ e v sono raggiungibili l'uno dall'altro in G se e solo sono raggiungibili l'uno dall'altro in G^T)

- $G^T = (V, E^T)$
- V ha un vertice per ogni SCC in G
- E^T ha un arco se c'è un arco tra i corrispondenti SCC in G



Nel nostro esempio

Lemma: G^T è un dag. Più formalmente, siano $.$ e v due SCC distinti in G , siano $,v \in , v \in v$ e si supponga che ci sia un cammino $\rightsquigarrow$ in G . Allora non ci potrà essere un cammino $v \rightsquigarrow v$ in G^T

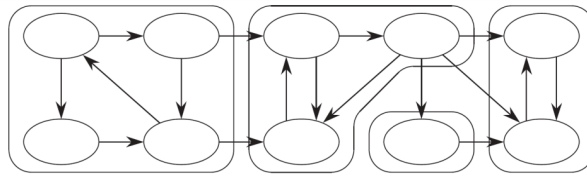
SCC(G)

chiama DFS(G) per calcolare i tempi di fine u.f per ogni vertice u

crea G^T

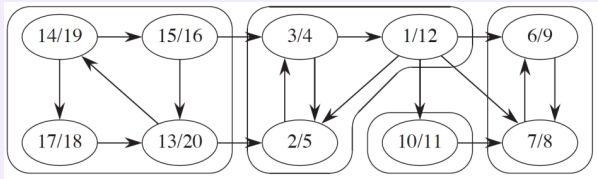
chiama DFS(G^T), ma nel loop principale considera i vertici in ordine decrescente di u.f (come valutato nel primo DFS)

restituisce i vertici in ogni albero del depth-first tree formato nel secondo DFS come un SCC separato
(?) esegue DFS in G (?)

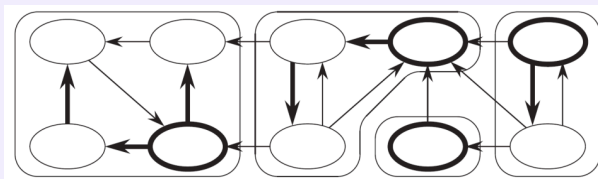


Example

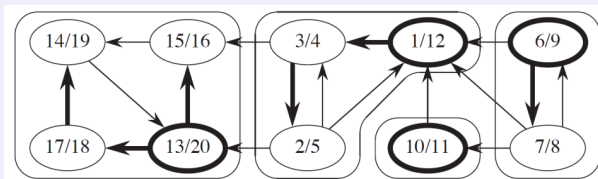
Eseguo DFS:



Prendo G :



DFS in G :

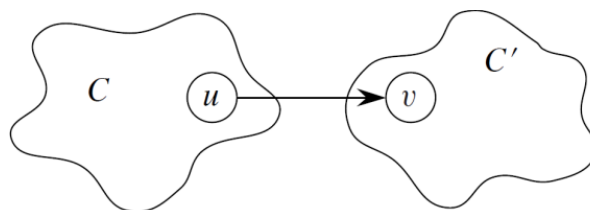


Tempo: $\Theta(V + E)$, ogni top level della DFS-Visit scoprirà il suo componente

Come può funzionare? considerando i vertici nel secondo DFS in ordine decrescente di tempi di fine (presi dal primo DFS), visitando i vertici del grafo del componente in ordine topologico. Per provare che ciò funziona affrontiamo 2 problemi notazionali saranno:

1. Discutere di d e f , questi si riferiranno sempre al primo DFS
2. Estendere la notazione per d e f per i set di vertici V :
 - $d() = \min\{d: \in\}$ (il primo discovery time in U)
 - $f() = \max\{f: \in\}$ (l'ultimo finish time in U)

Lemma: sia C e C' due SCC distinti in $G = (V, E)$. Si supponga che ci sia un arco $(u, v) \in E$ tale che $u \in C$ e $v \in C'$



Visitando i vertici di G in ordine inverso rispetto all'ordinamento topologico

Disjoint-set data structures

Le strutture dati per insiemi disgiunti (o union find) mantengono la collezione $\mathcal{C} = \{C_1, \dots, C_k\}$ di insiemi disgiunti e dinamici. Ogni set è identificato da un rappresentante, il quale è un qualche membro

dell'insieme. Non importa quale membro sia il rappresentante, fintanto che, se si chiede due volte quale sia il rappresentante, si ottenga la stessa risposta (senza cambiare il set)

Operazioni:

- **Make-Set(x)**: crea un nuovo set $i = \{x\}$ e aggiunge i a
- **Union(x, y)**: se $x \in x, \in$, allora $= -x - \{x\}$
 - Il rappresentante di ogni nuovo insieme è un membro di x (spesso il rappresentante o di x o di y)
 - Distrugge x e y (siccome gli insiemi devono essere disgiunti)
- **Find-Set(x)**: ritorna il rappresentante del set contenente x

L'analisi viene fatta in termini di:

- n = numero di elementi = numero di operazioni Make-Set
- m = numero totale di operazioni
- Siccome Make-Set conta fino al numero totale di operazioni, $m \geq n$
- Si possono avere al massimo $n-1$ operazioni Union, dato che rimane un solo insieme
- Si assuma che le prime n operazioni sono Make-Set (utile nell'analisi ma non necessario)

Per quanto riguarda l'applicazione ho dei componenti dinamici e connessi. Per un grafo $G = (V, E)$ i vertici u, v sono gli stessi componenti connessi se e solo se c'è un cammino tra di loro (i componenti connessi suddividono i vertici in classi di equivalenza)

```
Connected-Components(G)
  for ogni vertice v in G.V
    Make-Set(v)
  for ogni arco (u,v) in G.E
    if Find-Set(u) != Find-Set(v)
      Union(u,v)

Same-Components(u,v)
  if Find-Set(u) == Find-Set(v)
    return TRUE
  else
    return FALSE
```

Nota:

Se vengono implementate delle componenti connesse

- ogni vertice necessita di un gestore per il proprio oggetto nella struttura dati a insiemi disgiunti
- ogni oggetto nella struttura dati a insiemi disgiunti necessita di un gestore per il proprio vertice

Sintesi:

stai gestendo una partizione di elementi in gruppi, e vuoi poter dire velocemente "a quale gruppo appartiene questo elemento?" e "unisci questi due gruppi". Questo è esattamente il tipo di struttura che poi serve negli algoritmi sui grafi, soprattutto quando devi capire se due vertici sono già nello stesso componente.

Prima rappresentazione: linked list: ogni insieme è una linked list rappresentata da un oggetto con attributi:

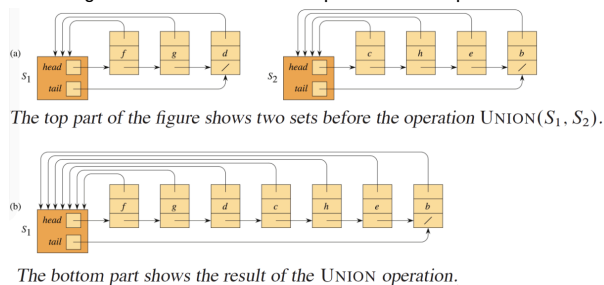
- *Testa*: il primo elemento nella lista, assumendo che sia il rappresentante dell'insieme
- *Coda*: l'ultimo elemento nella lista

Gli oggetti potrebbero apparire nella lista in qualsiasi ordine. Inoltre ogni oggetto della lista ha come attributi:

- Il membro dell'insieme
- Il puntatore all'oggetto "insieme"
- Next

Operazioni linked lists:

- **Make-Set:** crea una lista di singleton
- **Find-Set:** segue il puntatore fino all'oggetto lista e poi segue il puntatore testa fino al rappresentante
- **Union:** ci sono 2 modi per farla
 1. Appende la lista alla fine della lista x e usa il puntatore coda di x per trovare la fine. Quest'operazione necessita di aggiornare il puntatore all'oggetto insieme per ogni nodo nella lista. Chiaramente se appendo una lista grande su una lista piccola ci può volere molto tempo



2. **Weighted-union heuristic:** appende sempre la lista più piccola alla lista più grande

Teorema: con l'unione pesata una sequenza di operazioni su n elementi richiede $O(n \ln n)$ di tempo

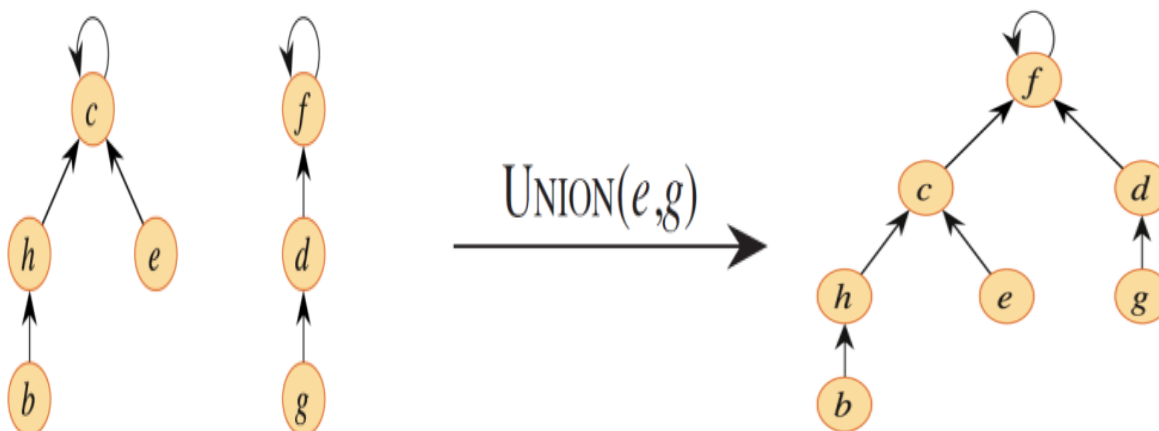
Dimostrazione: sia Make-Set che Find-Set richiedono $O(1)$, quante volte ogni rappresentante dell'oggetto (= la testa della lista) può essere aggiornato? Deve essere ogni volta nel set più piccolo. Perciò ogni rappresentante viene aggiornato $\leq \ln(n)$ volte

In altre parole: ogni elemento cambia "contenitore grande" solo un numero limitato di volte, perché ogni volta che viene spostato finisce in una lista almeno doppia rispetto a prima. Quindi il numero totale di aggiornamenti resta logaritmico per elemento.

Seconda rappresentazione: foresta di alberi: si tratta di una foresta di alberi

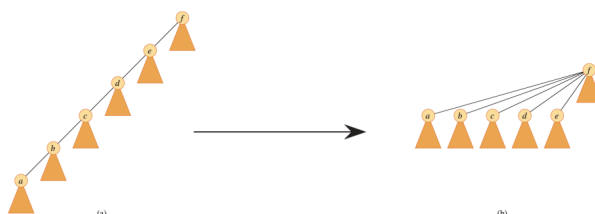
- Un albero per ogni set e la radice ne è il rappresentante
- Ogni nodo punta solo al proprio genitore
- **Make-Set:** crea un albero con un singolo nodo
- **Union:** fa sì che una radice sia figlia dell'altra
- **Find-Set:** segue i puntatori fino alla radice

Senza accorgimenti, questa struttura può degenerare in una catena lunga, quindi le operazioni possono diventare lente



Great heuristics:

- **Path compression:** durante Find-Set, tutti i nodi visitati vengono ricollegati direttamente alla radice.



- **Union by rank:** rende la radice dell'albero più piccolo (con meno nodi) un figlio della radice dell'albero più grande. Non utilizza la dimensione, ma il *rank* (che è un upper bound sull'altezza del nodo). Rende la radice con il grado più piccolo in un figlio della radice con un grado più grande

```

Make-Set
  x.p = x
  x.rank = 0

Union(x,y)
  Link(Find-Set(x),Find-Set(y))

Link(x,y)
  if x.rank > y.rank
    y.p = x
  else x.p = y
    if x.rank == y.rank
      y.rank = y.rank+1

Find-Set(x)
  if x != x.p //non è la radice
    x.p = Find-Set(x.p) //la radice diventa il genitore
  return x.p    //ritorna la radice

```

Running time: se si usano entrambe le union (per grado e compressione del cammino), $O((n))$. (n) è una funzione di crescita molto lenta. Il limite è stretto quindi esiste una sequenza di operazioni che richiede $((n))$ in tempo

Minimum spanning trees

Problema ipotetico:

- Una città ha un set di strade e un set di case
- Una strada connette 2 sole case
- Un a strada che connette le case e e v ha un costo di riparazione $w(e,v)$

L'obiettivo è di riparare abbastanza strade in modo da:

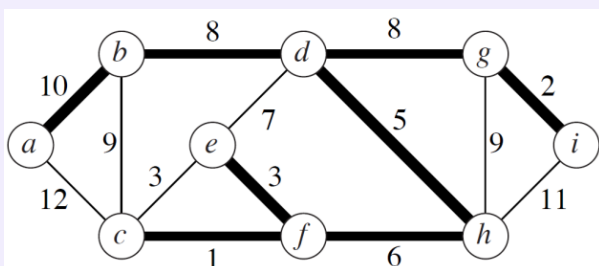
1. Tutti rimangano connessi (ogni casa può essere raggiunta da ogni altra casa)
2. Il costo di riparazione sia il minimo possibile

Modelliamo un grafo non orientato $G=(V,E)$ in cui ogni arco $(v,v) \in E$ abbia un peso $w(v,v)$. Devo trovare un E tale che:

1. connetta tutti i vertici (è uno spanning tree)
2. $w() = \sum_{(v,v) \in E} w(v,v)$ sia minimizzato

Uno spanning tree il cui peso sia minimo (tra tutti gli spanning trees) è chiamato (minimum spanning tree) o MST

≡ Example



In questo esempio c'è più di un MST. Ad esempio se rimpiazzassi l'arco (e,f) con l'arco (c,e) avrei un MST diverso con lo stesso peso

Proprietà di un MST:

- Ha $|V| - 1$ archi
- Non ha cicli
- Potrebbe essere unico come non

In sostanza:

Un minimum spanning tree (MST) è uno spanning tree di peso minimo in un grafo non orientato e pesato. Deve quindi:

- connettere tutti i vertici;
- avere esattamente $V - 1$ archi;
- non contenere cicli;
- avere peso totale minimo tra tutti gli spanning tree possibili.

Costruzione: si costruisce un set A di archi che inizialmente è vuoto. Man mano che vengono aggiunti archi ad A , si mantiene un'invariante del ciclo: A è il sottoinsieme di un qualche MST. Si possono aggiungere solo archi che mantengono l'invariante. Se A è un sottoinsieme di un MST, un arco (u,v) è sicuro per A se e solo se $A \cup \{(u,v)\}$ è a sua volta un sottoinsieme di un qualche MST.

Algoritmo generico MST:

```
Generic-MST(G,w)
  A =  $\emptyset$  //è vuoto all'inizio
  while A non forma uno spanning tree
    trova un arco  $(u,v)$  che sia sicuro per A
    A = A  $\cup \{(u,v)\}$ 
  return A
```

Per mostrare che questo algoritmo funziona usiamo il loop invariant:

- **Inizializzazione:** il set vuoto soddisfa il loop invariant
- **Mantenimento:** siccome solo archi validi sono aggiunti, A rimane un sottoinsieme di un qualche MST
- **Terminazione:** il loop deve terminare quando ha considerato tutti gli archi. Tutti gli archi inseriti dentro ad A sono parte di un MST. Alla fine A è uno spanning tree che è anche un MST

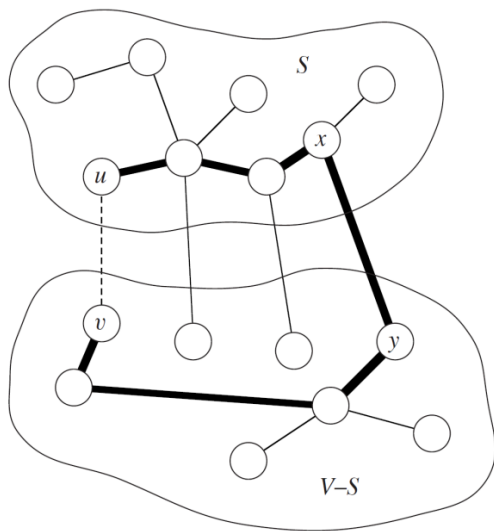
Come si trova un arco sicuro? guardando all'esempio l'arco (c,f) ha il peso minore di ogni altro arco nel grafo. È sicuro per A ? Sia V un sottoinsieme di vertici che include c ma non f ($f \in V$), è giusto scegliere l'arco con il peso minimo?

Definizioni:

- Un **taglio** $,V$ è una partizione di vertici negli insiemi disgiunti V e V
- L'arco $(,v) \in E$ attraversa il taglio $,V$ se un endpoint è in $$ e l'altro è in V
- Un taglio **rispetta** A se e solo se nessun arco di A attraversa il taglio
- Un arco è un **light edge** attraverso un taglio se e solo se il suo peso è minimo rispetto a tutti gli altri archi che attraversano il taglio. Dato un taglio, ci possono essere >1 light edge attraverso questo

Teorema: sia A un sottoinsieme di un qualche MST, $,V$ sia un taglio che rispetta A e sia $(,v)$ un light edge attraverso $,V$ $(,v)$ è sicuro per A

Dimostrazione:



Sia $(S, V \setminus S)$ una partizione rispetto ad A , un sottoinsieme di un qualche MST T . Tutti gli archi disegnati qui sono parte di T tranne (u,v) , il light edge del taglio. Se (u,v) non è nell'albero, rimpiazzando (x,y) con (u,v) da un nuovo spanning tree con un costo minore, una contraddizione!

Algoritmo di Kruskal: $G = (V, E)$ è un grafo connesso, non orientato e pesato. $w: E \rightarrow \mathbb{R}$

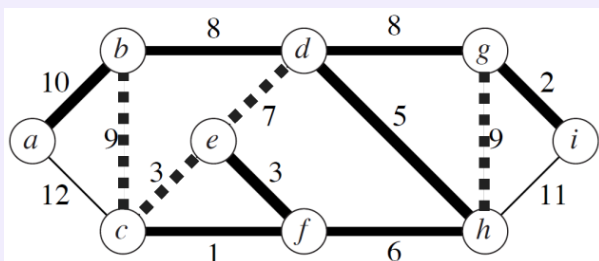
- Inizia con ogni vertice come componente separata e considera gli archi in ordine crescente di peso
- Fonde ripetutamente due componenti in uno scegliendo il light edge che li connette (prende un arco (u,v) : se i suoi estremi sono in componenti diverse, lo aggiunge all'insieme A e fa $\text{Union}(u,v)$; altrimenti lo scarta perché creerebbe un ciclo.)
- Usa una struttura dati basata su un insieme disgiunto per determinare quale arco connetta i vertici in componenti diversi

```

MST-Kruskal( $G, w$ )
   $A = \emptyset$ 
  for ogni vertice  $v$  in  $G.V$ 
    Make_Set( $v$ )
  crea una lista di archi in  $G.E$ 
  ordina la lista di archi in ordine crescente in base al peso  $w$ 
  for ogni arco  $(u,v)$  preso dalla lista ordinata
    if Find_Set( $u$ ) != Find_Set( $v$ )
       $A = A \cup \{(u,v)\}$ 
      Union( $u,v$ )
  return  $A$ 

```

≡ Vediamo l'algoritmo di Kruskal in questo grafo:



- archi accettati: $\{(c,f), (g,i), (e,f), (d,h), (f,h), (b,d), (d,g), (a,b)\}$
- archi rifiutati: $\{(c,e), (e,d), (b,c), (g,h)\}$

Analisi:

- Inizializzazione di A : $O(1)$
- Primo ciclo for: $|V|$ volte Make-Set
- Ordinare E : $O(E \ln(E))$
- Secondo ciclo for: $O(E)$ volte Find-Set e Union

Assumendo che l'implementazione di una struttura dati a insiemi disgiunti (disjoint-set), che usa l'unione in base al grado e compressione del percorso: $O((V + E)(V)) + O(E \ln(E))$.

- Siccome G è connesso, $|E| \geq |V| - 1$ $O(E(V)) + O(E \ln(E))$
- $(|V|) = O(\ln(V)) = O(\ln(E))$

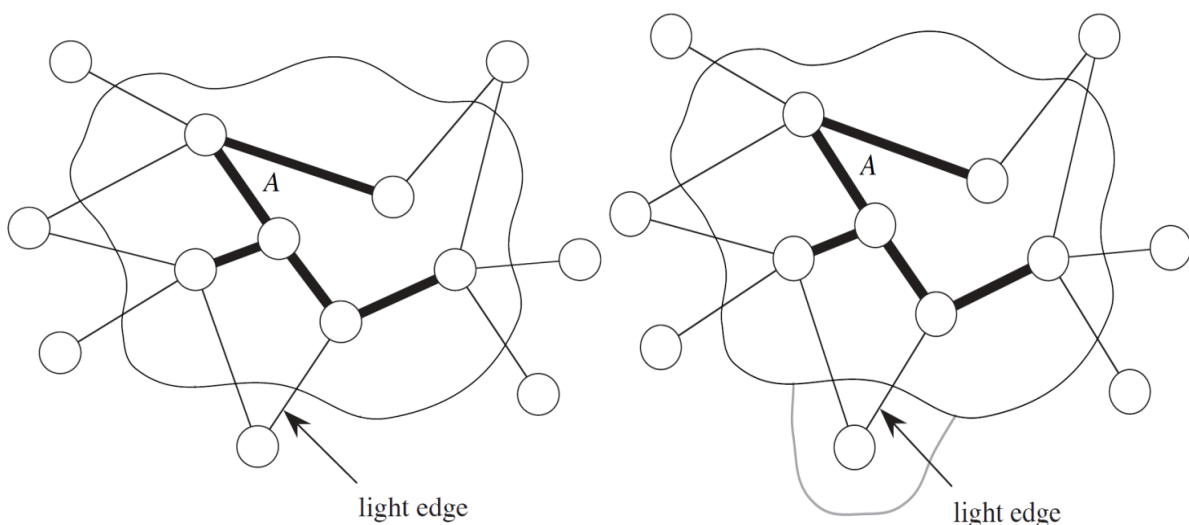
- Per questo motivo il tempo totale è $O(E \ln(E))$
- $E \leq |V|^2 \lg |E| = O(2 \ln(V)) = O(\ln(V))$
- Perciò il tempo è $O(E \ln(V))$ (se gli archi sono già ordinati $O(E(V))$ che è quasi lineare)

Sintesi:

Idea: aggiungi **gli archi più leggeri possibile**, evitando di creare cicli.

1. Ordina tutti gli archi per peso crescente
2. Inizializza una foresta (ogni nodo è un albero separato)
3. Per ogni arco (in ordine crescente):
 - se collega due componenti diverse → **aggiungilo**
 - se crea un ciclo → **scartalo**
4. Fermati quando hai $n-1$ archi (con n nodi)

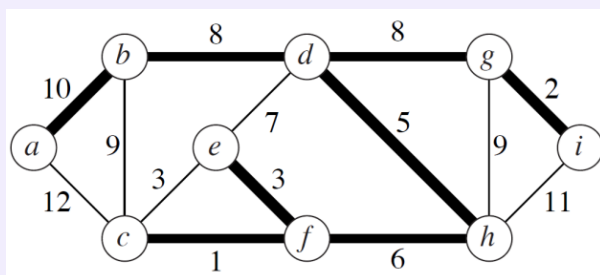
Algoritmo di Prim: Costruisce un solo albero alla volta, in questo modo A è sempre un albero e inizia da una "radice" arbitraria r . Ad ogni passo si trova un light edge che connette A ad un vertice isolato. Un arco, in questo modo, deve essere sicuro per A e può essere aggiunto.



Come trovo un **light edge**? devo usare una coda di priorità:

- Ogni oggetto è un vertice non presente in A che ha la proprietà $v.ke$
- $v.ke$ è il peso minimo di ogni arco che connette v ad un vertice in A . Se non esiste tale arco allora $v.ke = \infty$
- $v.\pi$ è il genitore di v in A
- Quando si trova un arco migliore si applica Decrease-Key
- Bisogna mantenere A implicitamente come $A = \{(v, v.\pi) : v \in V - \{r\} - \}$
- Una volta completato, è vuoto e il MST è $A = \{(v, v.\pi) : v \in V - \{r\}\}$

Algoritmo di Prim in questo grafo:



Analisi: dipende da come la coda di priorità viene implementata. Supponendo che sia un binary heap:

- Inizializzare e il primo ciclo for: $O(V \ln(V))$

- Diminuire la chiave di r : $O(\ln(V))$
- Ciclo while: $|V|$ chiamate di `Extract_Min` $O(V \ln(V)) \leq |E|$ chiamate di `Decrease_Key` $O(E \ln(V))$
- Totale: $O(E \ln(V))$

Si supponga che `Decrease-Key` possa richiedere $O(1)$ di tempo, allora $\leq |E|$ chiamate di `Decrease-Key` richiedono $O(E)$ in tempo tutte insieme tempo totale diventa $O(V \ln(V) + E)$

Differenza tra Kruskal e Prim:

- **Kruskal**: guarda gli archi uno per uno in ordine di peso e unisce componenti diverse. È "edge-driven".
- **Prim**: espande un unico albero da un vertice iniziale, aggiungendo ogni volta il miglior arco verso l'esterno. È "vertex/tree-driven".

Entrambi usano il criterio del taglio, ma in modo diverso: Kruskal lavora sui tagli tra componenti, Prim sul taglio tra l'albero già costruito e il resto del grafo.

Single-source shortest path

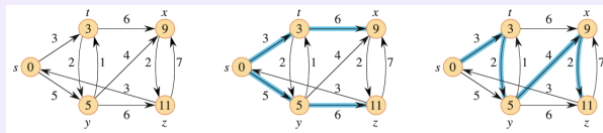
Come trovo il cammino minimo tra 2 punti su una mappa? Ho come input:

- Un grafo orientato $G = (V, E)$
- Una funzione per il peso $w: E \rightarrow \mathbb{R}$

Peso del cammino minimo da a a v : $\delta(a, v) = \begin{cases} \min_p \{w(p) : \text{se esiste un cammino } a \rightarrow v\} \\ \infty \end{cases}$ altrimenti

Il cammino minimo da a a v è un qualsiasi cammino tale che $w(p) = \delta(a, v)$

Cammino minimo da :



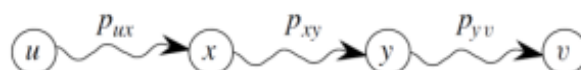
Questo esempio mostra che il cammino minimo potrebbe essere unico. Mostra anche che, quando si guarda il cammino minimo da un vertice a tutti gli altri, i cammini minimi sono organizzati come un albero

Va tutto bene finché nessun ciclo con costo negativo è raggiungibile dalla sorgente.

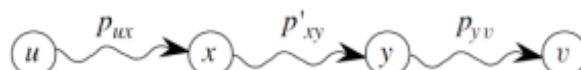
- Se abbiamo un ciclo con costo negativo, possiamo continuare attorno a questo e avere $w(v) = -\infty$ per ogni v nel ciclo
- Non ci sono problemi finché questo tipo di cicli non è raggiungibile
- Alcuni algoritmi lavorano solo se non ci sono archi con costo negativo nel grafo

Lemma: ogni sotto cammino di un cammino minimo è un cammino minimo

Prova: si usa la prova di correttezza taglia e incolla (*cut-and-paste*)



Si supponga che esista un cammino più breve p_x , allora $w(p_x) < w(p_x)$



Contraddice l'assunzione che p sia il cammino minimo

Cicli: un cammino minimo non può contenere cicli:

- Già trattati i cicli a costo negativo
- Se il costo è positivo possiamo ottenere un cammino minimo omettendo i cicli
- Costo 0 si assume che le nostre soluzioni non li useranno

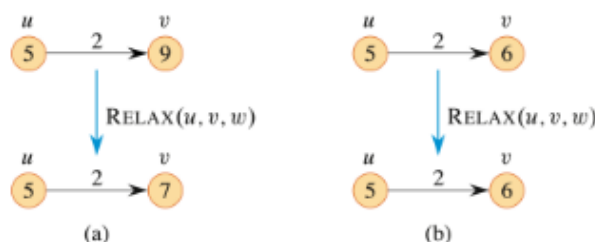
Output di un algoritmo per il cammino minimo con una singola sorgente: per ogni vertice $v \in V$

- $v.d = \delta(v)$
 - Inizialmente $v.d = \infty$
 - Si riduce con il progredire degli algoritmi, ma mantiene sempre $v.d \geq \delta(v)$
 - Chiama $v.d$ una stima di un cammino minimo
- $v.\pi$ = predecessore di v nel cammino minimo da
 - Se non ci sono predecessori, $v.\pi = NIL$
 - π induce un albero (shortest-path tree)

Inizializzazione: tutti gli algoritmi per i cammini minimi iniziano con:

```
Initialize-Single_source(G, s)
  For each vertex  $v \in G.V$ 
     $v.d = \infty$ 
     $v.\pi = NIL$ 
   $s.d = 0$ 
```

Funzione per rilassare un arco (u, v) : può la stima del cammino minimo per v essere migliorata andando attraverso u e prendendo (u, v) ?



Per ogni algoritmo per cammini minimi con una singola sorgente:

- Si inizia chiamando Initialize-Single-Source
- Poi si rilassano gli archi

Gli algoritmi differiscono tra di loro nell'ordine e quante volte vengono rilassati gli archi. Ciò significa verificare se passando da un certo nodo si trova un cammino più corto per arrivare a un altro nodo e, se sì, aggiornare la distanza.

Proprietà dei cammini minimi: basandosi sulla chiamata prima di Initialize-Single-Source e poi di Relax 0 o più volte:

- **Disuguaglianza triangolare:** per ogni $(u, v) \in E$, abbiamo che $\delta(u, v) \leq \delta(u, w) + w(u, v)$
- **Proprietà del limite superiore:** per ogni v si ha sempre che $v.d \geq \delta(v)$. Una volta che $v.d$ scende fino a $\delta(v)$, questo non cambia mai
- **Proprietà di no-path:** se $\delta(u, v) = \infty$, allora $v.d = \infty$ sempre
- **Proprietà di convergenza:** se $u \rightsquigarrow v$ è il cammino minimo, $u.d = \delta(u)$, e l'arco (u, v) è rilassato, quindi $v.d = \delta(v)$
- **Proprietà di path-relaxation:** sia $p = \langle v_0, v_1, \dots, v_k \rangle$ il cammino minimo da $u = v_0$ fino a v_k . Se gli archi di p sono rilassati, nell'ordine $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, anche se mescolati con altri rilassamenti, allora $v_k.d = \delta(v_k)$

Algoritmo di Bellman-Ford:

- Permette archi a costo negativo
- Calcola $v.d$ e $v.\pi$ per ogni $v \in V$
- Ritorna E se nessun ciclo di costo negativo sono raggiungibili da u , ALE altrimenti

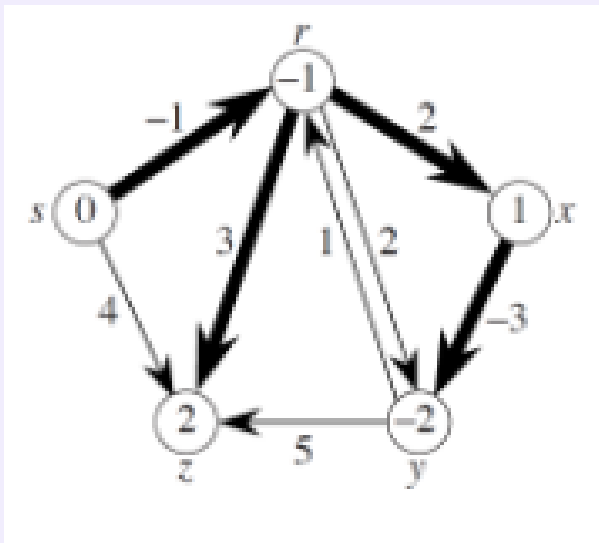
```

Bellman-Ford( $G, w, s$ )
    Initialize-Single-Source( $G, s$ )
    For  $i = 1$  to  $|G.V| - 1$ 
        For each vertex  $(u, v) \in G.E$ 
            Relax( $u, v, w$ )
    For each vertex  $(u, v) \in G.E$ 
        If  $v.d > u.d + w(u, v)$ 
            Return FALSE
    Return TRUE

```

Tempo: $O(V^2 + VE)$. Il primo ciclo for compie $|V| - 1$ passi sugli archi, e ogni passo richiede $\Theta(V + E)$ in tempo. Usiamo O piuttosto che Θ perché a volte sono sufficienti $< |V| - 1$

Example



Cammino minimo da una sorgente singola in un grafo diretto e aciclico: in un dag è garantito che non ci siano cicli a costo negativo

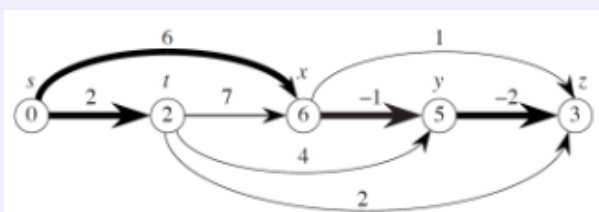
```

Dag-Shortest-Paths( $G, w, s$ )
    Topologically sort the vertices of  $G$ 
    Initialize-Single-Source( $G, s$ )
    For each vertex  $u \in G.V$ , preso in ordine
    topologicamente ordinato
        For each vertex  $v \in G.Adj[u]$ 
            Relax( $u, v, w$ )

```

- Tempo: $\Theta(V + E)$
- Correttezza: poiché i vertici sono elaborati in ordine topologicamente orientato, gli archi di ogni cammino devono essere rilassati in ordine di apparizione nel cammino gli archi su ogni cammino minimo sono rilassati in ordine per la proprietà di path-relaxation, è corretto

Example



Algoritmo di Dijkstra: non ci sono archi a costo negativo ed essenzialmente è una versione pesata del breadth-first-search:

- Al posto di una coda FIFO, usa una coda di priorità
- Le chiavi sono i pesi del cammino minimo ($v.d$)
- Può essere pensato in ondate come il BFS
- Un'onda viene emanata dalla sorgente
- La prima volta che un'onda arriva ad un vertice, una nuova onda viene emanata dal vertice

Ho due set di vertici:

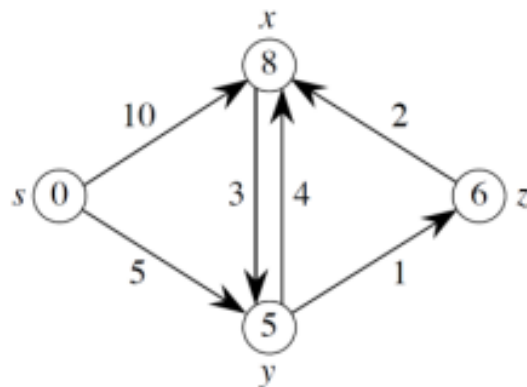
- = i vertici i cui pesi del cammino minimo finali sono determinati
- = coda di priorità = $V -$

```
Dijkstra( $G, w, s$ )
  Initialize-Single-Source( $G, s$ )
   $S = \emptyset$ 
   $Q = \emptyset$ 
  For each vertex  $u \in G.V$ 
    Insert( $Q, u$ )
  While( $Q \neq \emptyset$ )
     $u = \text{Extract-min}(Q)$ 
     $S = S \cup \{u\}$ 
    For each vertex  $v \in G.Adj[u]$ 
      Relax( $u, v, w$ )
      If the call of Relax decreased  $v.d$ 
        Decrease-Key( $Q, v, v.d$ )
```

- Sembra molto come l'algoritmo di Prim, ma calcola $v.d$ e usa i pesi del cammino minimo come chiave
- L'algoritmo di Dijkstra può essere visto come "avidio" poiché sceglie sempre il vertice "più leggero" ("più vicino?") in $V -$ da aggiungere a

Correttezza: l'algoritmo estrae i vertici dall'heap in base alla distanza minore dalla sorgente.

Induttivamente se l'algoritmo ha trovato il cammino minimo per qualche insieme, il cammino minimo per arrivare al vertice più vicino in $V -$ può essere trovato appendendo un solo arco ad un cammino a qualche vertice di



Order of adding to S : s, y, z, x .

Analisi: ho $|V|$ operazioni Insert e Extract-Min e $\leq |E|$ operazioni Decrease-Key. Come nell'algoritmo di Prim dipende dall'implementazione di una coda di priorità:

- Nel caso di una binary heap ogni operazione richiede $O(1)$ in tempo $O(E \lg(V))$
- Nel caso di una Fibonacci heap:
 - Ogni Extract-Min richiede $O(1)$ di tempo ammortizzato
 - Ci sono $\Theta(V)$ operazioni Insert e Extract-Min, che richiedono $O(\lg(V))$ di tempo ammortizzato
 - Perciò il tempo è $O(V \lg(V + E))$

All-Pairs shortest path

Dato un grafo orientato $G = (V, E)$ e una funzione peso $w: E \rightarrow \mathbb{R}, |V| = n$. Si assume che i vertici siano numerati $1, 2, \dots, n$. Il mio obiettivo è di creare una matrice $n \times n$ $D = (d_{ij})$ di shortest-paths distances, in questo modo $d_{ij} = \delta(i, j)$ per ogni vertice i e j . Si potrebbe eseguire l'algoritmo di Bellman-Ford per ogni vertice e avrebbe costo $O(V^2E)$, che è $O(V^4)$ se il grafo è **denso** ($E = \Theta(V^2)$). Se non ci sono archi a peso negativo, si potrebbe eseguire l'algoritmo di Dijkstra una volta per ogni vertice. In tal caso avrei:

- $O(VE \lg(V))$ con una binary heap, $O(V^3 \lg(V))$ se è denso
- $O(V^2 \lg(V) + VE)$ con una Fibonacci heap, $O(V^3)$ se è denso

Si assuma che il grafo G sia dato come matrice di adiacenza di pesi: $W = (w_{ij})$ con i vertici numerati da 1 a n

$$\Rightarrow w_{ij} = \begin{cases} 0 & \text{se } i = j \\ \text{peso dell'arco } (i, j) & \text{se } i \neq j, (i, j) \in E \\ \infty & \text{se } i \neq j, (i, j) \notin E \end{cases}$$

Soluzione ricorsiva: sia $l_{ij}^{(r)}$ = peso del cammino minimo $i \rightsquigarrow j$ che contiene $\leq r$ archi.

- $r = 0$: c'è un cammino minimo $i \rightsquigarrow j$ con $\leq r$ archi se e solo se $i = j$. Pertanto $l_{ij}^{(0)} = \begin{cases} 0 & \text{se } i = j \\ \infty & \text{se } i \neq j \end{cases}$
- $r \geq 1$: quindi $l_{ij}^{(r)} = \min\{l_{ij}^{(r-1)}, \min\{l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n\}\} = \min\{l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n\}$ con k i range con ogni possibile predecessore di j e sapendo che $w_{jj} = 0 \forall j$
- Si osserva che quando $r = 1$ si ha che $l_{ij}^{(1)} = w_{ij}$. Concettualmente quando il cammino è ristretto ad un solo arco, il peso del cammino minimo $i \rightsquigarrow j$ deve essere w_{ij}

Calcolo di una soluzione bottom-up (metodo tropicale): si calcolano $L^{(1)}, L^{(2)}, \dots, L^{(n)}$. Si inizia da $L^{(1)} = W$ siccome $l_{ij}^{(1)} = w_{ij}$, andando da $L^{(r-1)}$ a $L^{(r)}$

```
Extend-Shortest-Paths( $L^{(r-1)}, W, L^{(r)}, n$ )
//si assume che gli elementi di  $L^{(r)}$  sono inizializzati a infinito
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
             $l_{ij}^{(r)} = \min\{l_{ij}^{(r)}, l_{ik}^{(r-1)} + w_{kj}\}$ 
```

Il tempo di questo algoritmo è $\Theta(n^3)$.

Valutando ogni $L^{(r)}$:

```
Slow-APSP( $W, L^{(0)}, n$ )
let  $L = (l_{ij})$  and  $M = (m_{ij})$  be new  $n \times n$  matrices
 $L = L^{(0)}$ 
for r = 1 to n-1
     $M = \infty$  //initialize M
    // Compute the matrix product  $M = LW$ 
    Extend-Shortest-Paths( $L, W, M, n$ )
     $L = M$ 
return L
```

Il tempo di questo algoritmo è $\Theta(n^4)$.

Osservazione: Extend-Shortest-Paths è simile ad una moltiplicazione matriciale. Facendo, infatti, le seguenti sostituzioni nell'equazione $l_{ij}^{(r)} = \min\{l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n\}$:

- $l^{(r-1)} \rightarrow a$
- $w \rightarrow b$
- $l^{(r)} \rightarrow c$
- $\min \rightarrow +$
- $+ \rightarrow \cdot$

Si ottiene $c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$, che è proprio l'equazione per calcolare c_{ij} in una moltiplicazione matriciale. Se inoltre rimpiazzo ∞ (identità per $\min$) con 0 (identità per $+$), si ottiene la procedura per la moltiplicazione matriciale:

```
for i = 1 to n
    for j = 1 to n
        for k = 1 to n
             $c_{ij} = c_{ij} + (a_{ik})(b_{kj})$ 
```

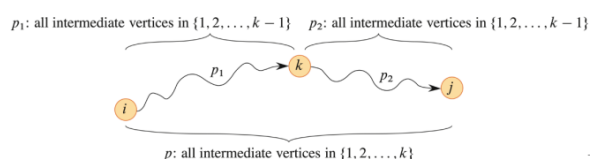
Siccome il nostro obiettivo è quello di valutare il più velocemente possibile $L^{(r-1)}$, non mi è necessario calcolare tutti gli $L^{(1)}, L^{(2)}, \dots, L^{(n-2)}$ intermedi. Si supponga di avere una matrice A e vogliamo calcolare $A^{(n-1)}$ (come chiamare Extend-Shortest-Paths $n-1$ volte). Si calcolano $A, A^2, A^4, A^8, \dots$, se sapessimo che $A^e = A^{(n-1)} \forall r \geq n-1$ si potrebbe finire con A^r , dove r è la più piccola potenza di 2 che è $\geq n-1$ ($r = 2^{\lceil \lg(n-1) \rceil}$)

```
Fast-APSP(W, n)
  let L and M be new  $n \times n$  matrices
  L = W
  r = 1
  while r < n-1
    M =  $\infty$  //initialize M
    Extend-Shortest-Paths(L, L, M, n) //compute  $M = L^2$ 
    r = 2r
    L = M //ready for the next iteration
  return L
```

Il tempo di questo algoritmo è $\Theta(n^3 \lg(n))$.

Algoritmo di Floyd-Warshall: si tratta di un approccio di programmazione diverso. Per il cammino $p = \langle v_1, v_2, \dots, v_l \rangle$ un vertice intermedio è un qualsiasi vertice di p diverso da v_1 o v_l . Sia $d_{ij}^{(k)}$ il peso del cammino minimo di ogni cammino $i \rightsquigarrow j$ con tutti i vertici intermedi in $\{1, 2, \dots, k\}$:

- Se k non è un vertice intermedio, allora tutti i vertici intermedi di p sono in $\{1, 2, \dots, k-1\}$
- Se k è un vertice intermedio:



$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{se } k = 0 \\ \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{se } k \geq 1 \end{cases}$$
 Si ha $d_{ij}^{(0)} = w_{ij}$ perché non si possono avere vertici intermedi $\Rightarrow \leq 1$ archi.

Si vogliono $D^{(N)} = (d_{ij}^{(n)})$ siccome tutti i vertici numerati sono $\leq n$

```
Floyd-Warshall(W, n)
  D(0) = W
  for j = 1 to n
    let D(k) = (dij(k)) be a new  $n \times n$  matrix
    for i = 1 to n
      for j = 1 to n
        dij(k) = min{dij(k-1), dik(k-1) + dkj(k-1)}
  return D(n)
```

Il tempo di questo algoritmo è $\Theta(n^3)$.

Calcolare i predecessori: è possibile valutare la matrice dei predecessori Π mentre si calcolava la matrice D . Sia $\Pi^{(k)} = (\pi_{ij}^{(k)})$, $k = 0, 1, \dots, n$. Si definisce $\pi_{ij}^{(k)}$ ricorsivamente. Per $k = 0$, un cammino minimo da i a j non ha vertici intermedi: $\pi_{ij}^{(0)} = \begin{cases} NIL & \text{se } i = j \text{ oppure } w_{ij} = \infty \\ i & \text{se } i \neq j \text{ e } w_{ij} < \infty \end{cases}$

Chiusura transitiva: dato un grafo orientato $G = (V, E)$, calcolo $G = (V, E)$ con $E = \{(i, j) : \text{se esiste un cammino } i \rightsquigarrow j \text{ in } G\}$. Si potrebbe assegnare un peso di 1 ad ogni arco e poi eseguire Floyd-Warshall:

- Se $d_{ij} < n$ allora c'è un cammino $i \rightsquigarrow j$
- Altrimenti $d_{ij} = \infty$ e non c'è un cammino tra i due nodi in questione

Una strada più semplice consiste nel sostituire valori e operatori in Floyd-Warshall:

- Uso una matrice delle adiacenze senza pesi
- Min $\rightarrow$ (OR)
- $\rightarrow$ (AND)
- $t_{ij}^{(k)} = \begin{cases} 1 & \text{se c'è un cammino } i \rightsquigarrow j \text{ con tutti i vertici intermedi in } \{1, 2, \dots, k\} \\ 0 & \text{altrimenti} \end{cases}$
- $t_{ij}^{(0)} = \begin{cases} 0 & \text{se } i \neq j \text{ e } (i, j) \notin E \\ 1 & \text{se } i = j \text{ oppure } (i, j) \in E \end{cases}$
- $t_{ij}^{(k)} = t_{ij}^{(k-1)} \text{ OR } (t_{ik}^{(k-1)} \text{ AND } t_{kj}^{(k-1)})$

TRANSITIVE-CLOSURE(G, n)

```

1  let  $T^{(0)} = (t_{ij}^{(0)})$  be a new  $n \times n$  matrix
2  for  $i = 1$  to  $n$ 
3      for  $j = 1$  to  $n$ 
4          if  $i == j$  or  $(i, j) \in G.E$ 
5               $t_{ij}^{(0)} = 1$ 
6          else  $t_{ij}^{(0)} = 0$ 
7  for  $k = 1$  to  $n$ 
8      let  $T^{(k)} = (t_{ij}^{(k)})$  be a new  $n \times n$  matrix
9      for  $i = 1$  to  $n$ 
10         for  $j = 1$  to  $n$ 
11              $t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$ 
12  return  $T^{(n)}$ 

```

Il tempo di questo algoritmo è $\Theta(n^3)$, ma con operazioni più semplici di Floyd-Warshall

Algoritmo di Johnson: se il grafo è sparso, questo paga per eseguire l'algoritmo di Dijkstra una volta per ogni vertice. Se usiamo la Fibonacci heap per la coda di priorità, il tempo di esecuzione sarebbe $O(V^2 \lg(V) + VE)$ che è meglio del $O(V^3)$ del Floyd-Warshall (se $E = (V^2)$). Ma l'algoritmo di Dijkstra richiede che tutti i vertici abbiano peso non negativo. Con l'algoritmo di Johnson si può costruire un grafo equivalente che non abbia archi di peso negativo. Per fare ciò ho bisogno di una funzione peso w tale che:

1. Per ogni $(v, p) \in E$, p è il cammino minimo $v \rightsquigarrow p$ usando w se e solo se p è il cammino minimo $v \rightsquigarrow p$ usando w
2. Per ogni $(v, v) \in E$, $w(v, v) \geq 0$

La prima proprietà afferma che è sufficiente trovare il cammino minimo con w . La seconda proprietà afferma che possiamo così eseguire l'algoritmo di Dijkstra da ogni vertice

Lemma (il reweighting non cambia il cammino minimo): dato un grafo diretto e pesato $G = (V, E), w: E \rightarrow \mathbb{R}$. Sia h una qualsiasi funzione tale che $h: V \rightarrow \mathbb{R}$. Per ogni $(v, v) \in E$ si definisce $w(v, v) = w(v, v) + h(v) - h(v)$. Sia $p = \langle v_1, v_2, \dots, v_l \rangle$ un qualsiasi cammino $v_0 \rightsquigarrow v_k$, allora p è un cammino minimo $v_0 \rightsquigarrow v_k$ con w se e solo se G non ha cicli a peso negativo con w .

Dimostrazione: per prima cosa si dimostra che $w(p) = w(p) + h(v_0) - h(v_k)$:

$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i) = \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)) = \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_0) - h(v_k)) = w(p) + h(v_0) - h(v_k)$. Così ogni cammino $v_0 \xrightarrow{p} v_k$ ha $w(p) = w(p) + h(v_0) - h(v_k)$ e $h(v_k)$ non dipende dal cammino a partire da v_0 fino a v_k . Se un cammino $v_0 \rightsquigarrow v_k$ è più breve di un altro con w , è anche più corto con w .

Ora è necessario ottenere una funzione $h: V \rightarrow \mathbb{R}$ così che quando valutiamo $w(v, v) = w(v, v) + h(v) - h(v)$ è ≥ 0 . Si fa la stessa cosa per vincoli diversi:

- $G = (V, E)$
 - $V = V \cup \{s\}$ dove s è un nuovo vertice
 - $E = E \cup \{(s, v) : v \in V\}$
 - $w(s, v) = 0 \forall v \in V$

Si definisce $h(v) = \delta(s, v) \forall v \in V$

Claim: $w(v, v) = w(v, v) + h(v) - h(v) \geq 0$

Dimostrazione: dalla disuguaglianza triangolare, $\delta(s, v) \leq \delta(s, u) + w(u, v)$, $h(v) \leq h(u) + w(u, v)$. Perciò $w(u, v) + h(v) - h(u) \geq 0$

```

Johnson(G, w)
compute G', where G'.V = G.V ∪ s, G'.E = G.E ∪ {(s, v) : v ∈ G.V}, and w(s, v) = 0 for all v ∈ G.V
if Bellman-Ford(G', w, s) == FALSE
    print "the input graph contains a negative-weight cycle"
else
    for each vertex v ∈ G'.V
        set h(v) to the value of δ(s, v) computed by the Bellman-Ford algorithm
    for each edge (u, v) ∈ G'.E
        ŵ(u, v) = w(u, v) + h(v) - h(u)
    let D = (duv) be a new n × n matrix
    for each vertex u ∈ G'.V
        run Dijkstra(G, ŵ, u) to compute δ̂(u, v) for all v ∈ G.V
        for each vertex v ∈ G.V
            duv = δ̂(u, v) + h(v) - h(u)
    return D

```

Tempo di questo algoritmo:

- $\Theta(V + E)$ per calcolare G
- $O(VE)$ per eseguire Bellman-Ford

- $\Theta(E)$ per calcolare w
- $O(V^2 \lg(V) + VE)$ per eseguire l'algoritmo di Dijkstra $|V|$ volte (usando una Fibonacci heap)
- $\Theta(V^2)$ per valutare la matrice D

In totale il tempo speso per questo algoritmo è: $O(V^2 \lg(V) + VE)$